

CS8.401: PRINCIPLES OF INFORMATION SECURITY

Programming Assignments

Comprehensive Specification Explanation

Contents

1	Prelude: The Cryptographic Universe and the Great Clique	3
1.1	Motivation	3
1.2	The Cryptographic Primitive Hierarchy	3
1.3	Note	3
1.4	Concrete Instantiations	7
1.5	The No-Library Rule	7
1.6	Security Notions Roadmap	8
2	Part 0: The Minicrypt Clique Web Explorer (Interactive Component)	9
2.1	The Core Problem the App Must Solve	9
2.2	Layout Specification	10
2.3	The Under-the-Hood Requirement (Critical)	11
2.4	Supported Reductions and the Routing Table	11
2.5	PA #0 — What You Must Implement	12
2.6	PA #1 — One-Way Functions & Pseudorandom Generators	14
2.7	PA #2 — Pseudorandom Functions via GGM Tree	15
2.8	PA #3 — CPA-Secure Symmetric Encryption	17
2.9	PA #4 — Modes of Operation	19
2.10	PA #5 — Message Authentication Codes (MACs)	21
2.11	PA #6 — CCA-Secure Symmetric Encryption	22
3	Part II: Hashing and Data Integrity	25
3.1	PA #7 — Merkle-Damgård Transform	25
3.2	PA #8 — DLP-Based Collision-Resistant Hash Function	27
3.3	PA #9 — Birthday Attack (Collision Finding)	29
3.4	PA #10 — HMAC and HMAC-Based CCA-Secure Encryption	30

4	Part III: Public-Key Cryptography (Cryptomania)	34
4.1	PA #11 — Diffie-Hellman Key Exchange (SKE)	34
4.2	PA #12 — Textbook RSA	35
4.3	PA #13 — Miller-Rabin Primality Testing	37
4.4	PA #14 — Chinese Remainder Theorem & Breaking Textbook RSA	39
4.5	PA #15 — Digital Signatures	41
4.6	PA #16 — ElGamal Public-Key Cryptosystem	43
4.7	PA #17 — CCA-Secure PKC	44
5	Part IV: Secure Multi-Party Computation (MPC)	46
5.1	PA #18 — Oblivious Transfer (OT)	46
5.2	PA #19 — Secure AND Gate	48
5.3	PA #20 — All 2-Party Secure Computation (Yao / GMW)	49
6	Summary: The Full Reduction Chain	52

1 Prelude: The Cryptographic Universe and the Great Clique

1.1 Motivation

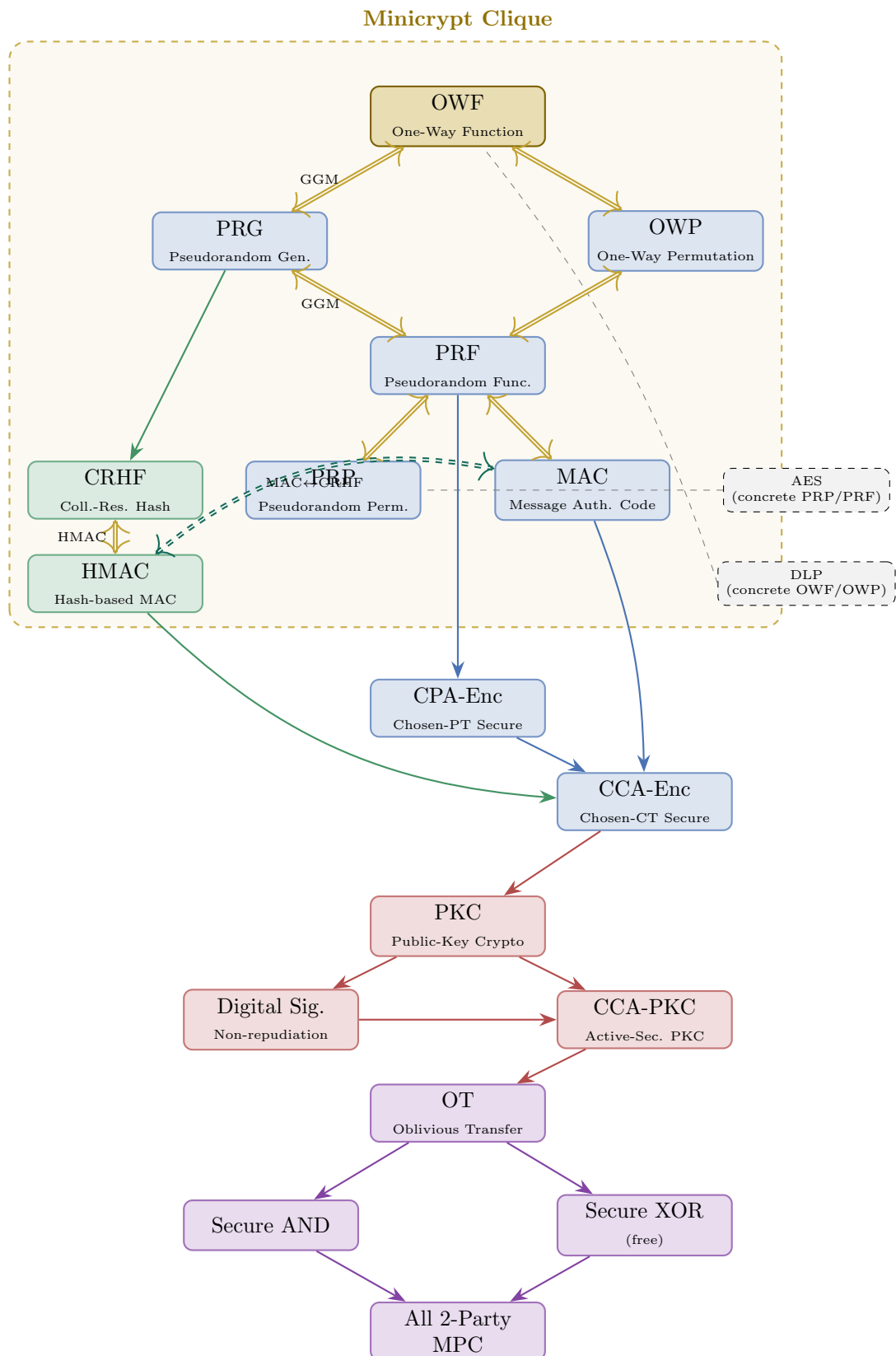
Modern cryptography is built on a beautifully minimal foundation: the existence of a **One-Way Function (OWF)**. From this single computational assumption—that some functions are easy to compute but hard to invert—we can derive *every* symmetric cryptographic primitive used in practice. This chain of reductions is both the theoretical backbone of our assignments and a profound statement about what computation can and cannot do.

1.2 The Cryptographic Primitive Hierarchy

The diagram below shows the *complete reduction graph* among the primitives we study. An arrow $A \rightarrow B$ means “ A implies B ” (i.e., given A we can construct B). The double-headed arrows show the remarkable **equivalences** at the core of Minicrypt.

1.3 Note

We have added comprehensive explanation of each programming assignment to avoid any confusion and assist easy implementations. If you have any doubts, feel free to reach out to your respective TAs.



The Minicrypt Equivalence Theorem

The following are **equivalent** (each implies all others):

- One-Way Functions (OWFs) exist
- Pseudorandom Generators (PRGs) exist
- Pseudorandom Functions (PRFs) exist
- One-Way Permutations (OWPs) exist
- Pseudorandom Permutations (PRPs) exist
- Secure Message Authentication Codes (MACs) exist
- Collision-Resistant Hash Functions (CRHFs) exist (*linked via HMAC*)
- HMAC exists (*bridges CRHF $\leftrightarrow$ MAC*)

This is the *Minicrypt Clique*. Proving any one of these exists immediately gives you all the others—and your assignments will implement every step of this chain. Note that construction of all edges may not be possible since some might be open problems. Try to complete the ones that are already known.

Note on CRHF and HMAC: The classical Minicrypt clique consists of the first six primitives. CRHF and HMAC join the clique through HMAC's security proof: if the hash's compression function is a PRF (which our PA#8 DLP-based hash satisfies), then HMAC is a secure MAC ($CRHF \Rightarrow MAC$). The reverse direction holds because a secure MAC used as a Merkle-Damgrd compression function gives a CRHF ($MAC \Rightarrow CRHF$). This bidirectional link is demonstrated concretely in PA#9 (birthday attack, establishing the security floor of any CRHF) and PA#10 (HMAC construction + CCA-secure encryption).

Implementation Requirement: Bidirectional Reductions (All Clique Primitives)

The equivalences in the Minicrypt Clique are **biconditional**: $A \Leftrightarrow B$, not merely $A \Rightarrow B$. **Rule:** You are required to implement a primitive **if and only if** you also implement the reduction in *both* directions for every adjacent pair in the clique. For each pair (A, B) below you must demonstrate: (i) how to *construct* B from A , and (ii) how breaking B would allow you to *break* A (i.e., a reduction in the reverse direction). The complete set of required bidirectional implementations is:

OWF $\Leftrightarrow$ PRG

- *Forward* OWF $\Rightarrow$ PRG: HILL/iterative construction — apply f repeatedly with hard-core predicate b ; output $b(f^0(x)) \| b(f^1(x)) \| \dots$
- *Backward* PRG $\Rightarrow$ OWF: Any PRG G is immediately a OWF. Define $f(s) = G(s)$; inversion of f would invert G , recovering the seed and breaking pseudorandomness.

OWF $\Leftrightarrow$ OWP

- *Forward* OWF $\Rightarrow$ OWP: Any OWF on a domain with efficiently samplable pre-images can be converted to a OWP (e.g., DLP: $f(x) = g^x \bmod p$ is a OWP on $\mathbb{Z}_q$).
- *Backward* OWP $\Rightarrow$ OWF: Immediate — a OWP is a special case of a OWF (bijective, hence also hard to invert).

PRG $\Leftrightarrow$ PRF

- *Forward* PRG $\Rightarrow$ PRF: GGM tree construction. Given $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, define $F_k(b_1 \cdots b_n) = G_{b_n}(\cdots G_{b_1}(k))$.
- *Backward* PRF $\Rightarrow$ PRG: Define $G(s) = F_s(0) \| F_s(1)$. If G were distinguishable from random, the distinguisher could be used to break the PRF.

OWP $\Leftrightarrow$ PRG

- *Forward* OWP $\Rightarrow$ PRG: Any OWP with a hard-core predicate b yields a PRG: $G(x) = (f(x), b(x))$, expanding by one bit per application.
- *Backward* PRG $\Rightarrow$ OWP: A length-preserving PRG is itself a OWP (it must be injective and hard to invert, hence a permutation on its range).

PRF $\Leftrightarrow$ PRP

- *Forward* PRF $\Rightarrow$ PRP: Luby-Rackoff construction. Apply three or four rounds of a Feistel network using the PRF as the round function: a 3-round Feistel yields a secure PRP; a 4-round Feistel yields a secure *strong* PRP (indistinguishable even from an adversary with inversion queries).
- *Backward* PRP $\Rightarrow$ PRF: A PRP over a super-polynomially large domain is computationally indistinguishable from a PRF (by the PRF/PRP switching lemma). Concretely, AES (a PRP) is used directly as a PRF in CTR, OFB, and GCM.

PRF $\Leftrightarrow$ MAC

- *Forward* PRF $\Rightarrow$ MAC: $\text{Mac}_k(m) = F_k(m)$. If the MAC were forgeable, the forger distinguishes F_k from a random function, breaking PRF security.
- *Backward* MAC $\Rightarrow$ PRF: A secure EUF-CMA MAC on uniformly random messages is a PRF. Use the MAC oracle as a PRF oracle; unforgeability implies pseudorandomness of outputs.

PRP $\Leftrightarrow$ MAC (via PRF as bridge)

- *Forward* PRP $\Rightarrow$ MAC: Use the PRP directly as a PRF (switching lemma), then apply PRF $\Rightarrow$ MAC above. Concretely: AES-CMAC and CBC-MAC both use a block cipher (PRP) as the underlying primitive.
- *Backward* MAC $\Rightarrow$ PRP: Via MAC $\Rightarrow$ PRF (above) and then PRF $\Rightarrow$ PRP (Luby-Rackoff).

OWP $\Leftrightarrow$ PRF (completing the clique)

- *Forward* OWP $\Rightarrow$ PRF: OWP $\Rightarrow$ PRG (above) then PRG $\Rightarrow$ PRF (GGM).
- *Backward* PRF $\Rightarrow$ OWP: PRF $\Rightarrow$ PRP (Luby-Rackoff); a PRP on $\{0, 1\}^n$ keyed by k gives OWP $f(k) = \text{PRP}_k(0^n)$.

CRHF $\Leftrightarrow$ HMAC

- *Forward* CRHF $\Rightarrow$ HMAC: Given a hash function H built on a PRF-secure compression function (e.g., your PA#8 DLP hash), define $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m))$. Security holds because the inner hash acts as a PRF on the message, and the outer hash acts as a PRF on the inner hash output.

- *Backward* HMAC $\Rightarrow$ CRHF: Fix a key k and define $H'(m) = \text{HMAC}_k(m)$. This keyed function is collision-resistant (any collision would constitute a MAC forgery). More generally, use the HMAC compression step as the compression function in a new Merkle-Damgrd hash.

HMAC $\Leftrightarrow$ MAC (*HMAC joins the MAC equivalence class*)

- *Forward* HMAC $\Rightarrow$ MAC: HMAC is a secure EUF-CMA MAC when the compression function is a PRF. This is exactly what PA#10 implements.
- *Backward* MAC $\Rightarrow$ HMAC: Any secure PRF-based MAC can be cast in the HMAC double-hash structure by treating the MAC as the inner compression step. The HMAC structure is therefore not a special-case construction but the natural PRF-based MAC for hash-structured primitives.

CRHF $\Leftrightarrow$ MAC (*the full bridge, via HMAC*)

- *Forward* CRHF $\Rightarrow$ MAC: CRHF $\Rightarrow$ HMAC $\Rightarrow$ MAC (two steps above). Concretely: your PA#8 DLP hash $\Rightarrow$ PA#10 HMAC $\Rightarrow$ PA#10 CCA-secure encryption.
- *Backward* MAC $\Rightarrow$ CRHF: A secure MAC serves as a collision-resistant compression function (any collision in the MAC output is a forgery). Apply the Merkle-Damgrd transform (PA#7) to get a full CRHF.

This bidirectionality is what makes these six primitives a *clique*, not merely a chain. Submitting only forward constructions earns partial credit; full credit requires both directions for every pair you implement.

1.4 Concrete Instantiations

In practice we use two primary workhorses:

- **AES (Advanced Encryption Standard):** A concrete, highly optimized *PRP* (and by extension PRF) based on the algebraic structure of $\text{GF}(2^8)$. Whenever an assignment asks for a “generic PRF,” you may plug in AES.
- **Discrete Logarithm Problem (DLP):** The assumption that computing $\log_g h$ in a cyclic group $\mathbb{G}$ is computationally hard. This gives us a concrete OWF/OWP, and underpins Diffie-Hellman, ElGamal, and the DLP-based hash function.

1.5 The No-Library Rule

Global Policy: No External Cryptographic Libraries

Every cryptographic primitive used in any programming assignment **must be your own prior implementation**. You may *not* substitute a library (e.g. PyCryptodome, OpenSSL, BouncyCastle, `cryptography`, Java’s `javax.crypto`, etc.) at any point in the dependency chain.

Specifically:

- If PA#18 (OT) requires a PKC, you must supply your own implementation from

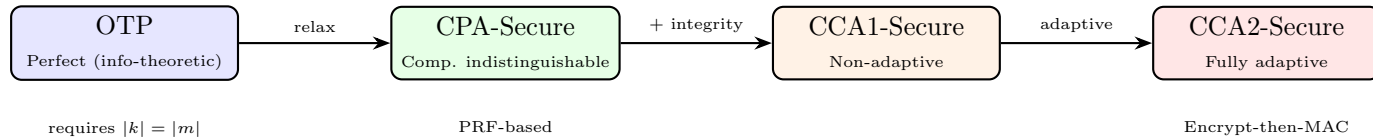
PA#12 or PA#16 — not `import rsa` or equivalent.

- If PA#6 or PA#10 (CCA-Enc) require a MAC, you must supply your own from PA#5 (for PRF-MAC) or PA#10 (for HMAC) — not HMAC from `hashlib` or any other library.
- If PA#10 (HMAC) requires a hash function, you must supply your own `DLP.Hash` from PA#8 — not SHA-256 or any standard library hash.
- If PA#4 (Modes) requires a block cipher, you must supply your own PRF/PRP from PA#2, or your own AES implementation — not a library AES call.
- PA#20 (MPC) must be built end-to-end using your PA#19 (Secure AND) and your PA#18 (OT) — not a garbled-circuit library.

The one permitted exception: You may use standard library functions for *arbitrary-precision integer arithmetic* (e.g. Python's built-in `int`, Java's `BigInteger`) and for *OS-level randomness* (`os.urandom`, `SecureRandom`). Everything above raw integers and random bytes must be your own code.

Why this rule exists: The entire point of these assignments is to trace the reduction chain in code. If you substitute a library primitive at any node, you break the chain and defeat the purpose of the exercise. The dependency graph below makes the required lineage explicit.

1.6 Security Notions Roadmap



2 Part 0: The Minicrypt Clique Web Explorer (Interactive Component)

What This Assignment Is and Why It Exists Here

The programming assignments in Parts I–IV ask you to implement each Minicrypt primitive on the command line. This section adds a complementary deliverable: a **React web application** that makes every reduction in the clique *visual, interactive, and traceable with real data*.

The app does not replace the terminal implementations. It *calls* them. Its purpose is to force you to think about a question the terminal assignments can obscure: **where does the concrete data actually come from?** Every abstract reduction in the clique says “given an oracle for primitive A , construct primitive B .” But an oracle is not code. You have to pick a concrete function—AES or DLP—and trace how that function flows all the way up the chain to B .

Placement: This section is described here, in the Prelude, because the app is built *incrementally* as you complete Parts I–IV. You can stub out primitives you have not yet implemented and fill them in as the course progresses. A fully working version is expected by the final submission deadline.

2.1 The Core Problem the App Must Solve

Consider the reduction $\text{PRG} \Rightarrow \text{PRF}$ (the GGM construction). Abstractly, it says: given any PRG G , define $F_k(x) = G_{x_n}(\dots G_{x_1}(k))$. But to run this on actual data you need an actual PRG. Where does it come from?

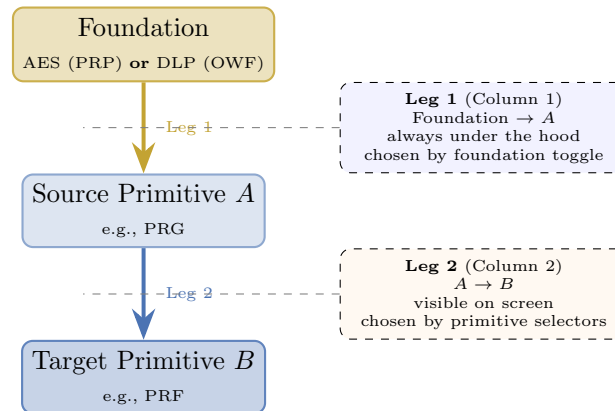
The answer is always one of two concrete starting points:

- **AES** — a concrete PRP/PRF. To get a PRG from AES: use AES as a PRF (PRP/PRF switching lemma), then define $G(s) = F_s(0) \| F_s(1)$.
- **DLP** — a concrete OWF/OWP. To get a PRG from DLP: apply the hard-core-bit construction (HILL), iterating $f(x) = g^x \bmod p$ to extract pseudorandom bits.

In either case, reaching PRF requires **two distinct legs**:

1. **Leg 1 (foundation $\rightarrow$ source primitive):** Instantiate the source primitive from the concrete foundation. This is always done under the hood, regardless of which two clique members the user picks.
2. **Leg 2 (source primitive $\rightarrow$ target primitive):** Apply the abstract reduction. This is the visible operation on screen.

The web app makes both legs explicit, with data flowing through each one.



2.2 Layout Specification

The app has a **fixed three-tier layout**:

1. **Top bar — Foundation selector:** A toggle with two options: AES-128 (PRP) and DLP ($g^x \bmod p$). Changing this toggle affects Column 1 entirely and re-runs all computations.
2. **Two-column main area:**
 - **Column 1 — “Build” panel (Leg 1: Foundation → Source Primitive A):**
 - A dropdown: select the *source primitive* A from the clique (OWF, PRG, PRF, PRP, MAC, CRHF, HMAC).
 - An input field: enter a raw key or seed (hex string).
 - A step-through display: shows each sub-reduction from the foundation to A , with the actual intermediate byte values at each step.
 - Example for AES + PRG: shows $\boxed{\text{AES key}} \xrightarrow{\text{PRF}} \boxed{F_k(0) \| F_k(1)} = \boxed{\text{PRG output}}$.
 - **Column 2 — “Reduce” panel (Leg 2: Source Primitive A → Target Primitive B):**
 - A dropdown: select the *target primitive* B (must differ from A).
 - An input field: enter the message or query to evaluate.
 - A step-through display: shows the reduction from A to B (e.g., GGM tree for PRG→PRF), with real intermediate values.
 - The output of Column 1 is automatically piped as the concrete implementation of A used in Column 2.
3. **Bottom panel — Reduction proof summary:** A collapsible box showing the full chain: Foundation → A → B , the theorem names (HILL, GGM, Luby-Rackoff, etc.), and the security claim at each step.

Foundation: AES-128 (PRP) / DLP ($g^x \bmod p$)
CS8.401 Minicrypt Clique Explorer

Column 1: Build Source Primitive from Foundation

Source primitive A : PRG ▼
Input seed: a3f2... (hex)

 $AES_k(0) = 7f3a...$
 $AES_k(1) = c19b...$
 $PRG(s) = 7f3ac19b...$

Column 2: Reduce Source to Target Primitive

Target primitive B : PRF ▼
Query x : 1011 (bits)

GGM: $G_1(G_0(k))$
 $= G_1(7f3a...)$
 $PRF_k(x) = 88d4...$

Reduction Chain Summary (click to expand)

$$AES \text{ (PRP)} \xrightarrow{\text{switching lemma}} PRF \xrightarrow{G(s)=F_s(0) \parallel F_s(1)} PRG \xrightarrow{\text{GGM tree}} PRF$$

Security: PRF-security of AES $\Rightarrow$ PRG-security (negl. advantage) $\Rightarrow$ PRF-security (GGM thm.)

All intermediate values shown above are real outputs from your PA#1–#2 implementations.

2.3 The Under-the-Hood Requirement (Critical)

Architectural Rule: The Foundation Must Flow Through Column 1

Column 2 is **not** allowed to call the foundation (AES or DLP) directly. It may only call the source primitive A that Column 1 has constructed.

Concretely: if the user selects Foundation = AES, Source = PRG, Target = PRF:

- **Correct:** Column 1 builds PRG using AES under the hood. Column 2 receives a PRG object and runs GGM on it, never touching AES directly.
- **Wrong:** Column 2 calls AES directly to implement the GGM tree leaves. This collapses the two legs into one and defeats the purpose.

This rule enforces the reduction structure: each primitive is a black box to the layer above it. The same rule applies to all choices of A and B .

2.4 Supported Reductions and the Routing Table

Not every pair (A, B) has a direct one-step reduction. The app must implement a **routing table** that computes the shortest path in the clique graph and chains the reductions:

Source A	Target B	Reduction chain used
OWF	PRG	HILL hard-core-bit construction
OWF	OWP	DLP is already a OWP (identity for DLP foundation)
PRG	PRF	GGM tree
PRF	PRP	Luby-Rackoff 3-round Feistel
PRF	MAC	$\text{Mac}_k(m) = F_k(m)$
PRP	MAC	PRP/PRF switching lemma, then MAC
CRHF	HMAC	HMAC construction (PA#10)
HMAC	MAC	Direct (HMAC is a MAC)
Any	Any	Compose the above steps as needed

For backward reductions ($B \rightarrow A$), the app also supports the reverse direction (bidirectional mode toggle).

2.5 PA #0 — What You Must Implement

PA #0 — What You Must Implement

1. **React app scaffold:** Create a React app (Create React App or Vite) with the three-tier layout described above. Foundation toggle, two-column main area, and bottom proof panel. Styling is your choice but the layout must be clearly two-column.
2. **Foundation layer:** Implement two foundation modules:
 - **AESFoundation:** wraps your PA#2 AES-based PRF. Exposes `asOWF()`, `asPRF()`, `asPRP()`.
 - **DLPFoundation:** wraps your PA#1 DLP-based OWF. Exposes `asOWF()`, `asOWP()`.

Both modules share a common **Foundation** interface so the rest of the app is agnostic.

3. **Column 1 — Build panel:** Given a Foundation and a target source primitive A , compute and display the full chain $\text{Foundation} \rightarrow A$, showing each intermediate value. Each step must display: the function applied, the input bytes (hex), and the output bytes (hex). The actual computation must call your existing PA implementations compiled to WebAssembly or via a local API — **not** reimplemented in JavaScript.
4. **Column 2 — Reduce panel:** Given the concrete instance of A produced by Column 1 and a target primitive B , compute and display the reduction $A \rightarrow B$ step by step. Column 2 must receive A as a black box (a function object) and must not inspect its internals.
5. **Routing table:** Implement `reduce(A, B, foundation)` that returns the ordered list of reduction steps to chain. Handle all pairs listed in the table above. For unsupported pairs (e.g., $\text{CRHF} \rightarrow \text{OWP}$), display a clear message explaining why no direct path exists in this direction and suggest using the bidirectional toggle.
6. **Live data flow:** When the user changes any input (foundation toggle, source/target primitive, key, message), all panels update in real time without a page reload.
7. **Bidirectional mode:** Add a toggle “Forward ($A \rightarrow B$) / Backward ($B \rightarrow A$)” that swaps the columns and shows the reverse reduction (e.g., $\text{MAC} \rightarrow \text{PRF}$ by querying the MAC as a PRF oracle and running the distinguishing game).
8. **Proof summary panel:** For each selected pair (A, B) , display the formal security chain: which theorem justifies each step, what the security reduction is (“if adversary breaks B with advantage ϵ , it breaks A with advantage $\epsilon' \geq \epsilon/q$ ”), and the PA number

that implements each step.

9. **Stub support:** Primitives you have not yet implemented should show a “Not implemented yet (due: PA#N)” placeholder with a greyed-out step in the chain. The app must be fully runnable with any subset of primitives implemented.

PA #0 — Interactive Demo Deliverable

The scaffold itself *is* the demo for PA#0. Graders will open the app in a browser and check:

- The foundation toggle (AES / DLP) is visible and switches without errors.
- Both columns render with their dropdowns and input fields.
- Selecting any primitive pair shows a “Not yet implemented” placeholder with the correct PA number — not a blank or crash.
- The bottom proof panel opens/closes on click and displays the static reduction chain text for the selected pair.

Toy parameters: No real crypto is needed for PA#0. Stub functions returning fixed hex strings are fine at this stage.

Intuition

Why start from scratch? A truly random One-Time Pad gives *perfect* (Shannon) secrecy but demands a key as long as the message. The key insight of computational cryptography is: we only need an adversary to be *computationally* indistinguishable from random, not information-theoretically so. A short random seed fed into a PRG yields a *pseudorandom* stream that is indistinguishable from true randomness by any efficient adversary. Upgrading to a PRF gives us indexed, non-sequential access, enabling MAC and encryption. This is the bootstrapping miracle of Minicrypt.

2.6 PA #1 — One-Way Functions & Pseudorandom Generators**One-Way Functions (OWFs)****Definition: One-Way Function**

A polynomial-time computable function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **one-way** if for every PPT adversary $\mathcal{A}$:

$$\Pr [\mathcal{A}(1^n, f(x)) \in f^{-1}(f(x))] \leq \text{negl}(n)$$

where $x \leftarrow \{0, 1\}^n$ uniformly at random.

Concrete OWFs to implement:

- **Modular exponentiation (DLP):** $f(x) = g^x \bmod p$ in a prime-order group.
- **Integer factoring:** $f(p, q) = p \cdot q$ for large primes p, q .
- **AES as a compression OWF:** $f(k) = \text{AES}_k(0^{128}) \oplus k$.

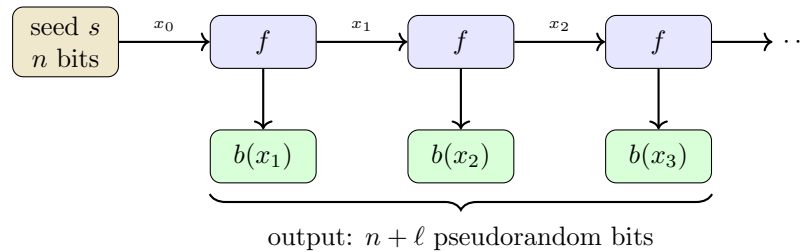
Pseudorandom Generators (PRGs)**Definition: PRG**

A deterministic polynomial-time function $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+\ell(n)}$ (for any $\ell(n) > 0$) is a **PRG** if for every PPT distinguisher D :

$$\left| \Pr_{s \leftarrow \{0, 1\}^n} [D(G(s)) = 1] - \Pr_{r \leftarrow \{0, 1\}^{n+\ell(n)}} [D(r) = 1] \right| \leq \text{negl}(n)$$

Construction of a PRG from a OWF (Håstad–Impagliazzo–Levin–Luby): Given OWF f with hard-core predicate b (e.g., the Goldreich-Levin bit), define:

$$G(x_0) = b(x_0) \parallel b(x_1) \parallel \cdots \parallel b(x_\ell) \quad \text{where } x_{i+1} = f(x_i)$$



Security Claim

If f is a OWF with hard-core predicate b , then G as constructed above is a secure PRG. Security reduces to the hardness of inverting f .

PA #1 — What You Must Implement

1. OWF (choose one concrete instantiation):

- DLP-based: $f(x) = g^x \bmod p$, with a safe prime p and generator g of prime-order subgroup.
- AES-based: $f(k) = \text{AES}_k(0^{128}) \oplus k$ (Davies-Meyer-style).
- Your OWF must expose a `evaluate(x)` function and a `verify_hardness()` demo (show that random inversion fails).

2. PRG from OWF (forward direction, PA#1a):

Implement the iterative hard-core-bit construction. Given seed $s \in \{0,1\}^n$, produce an output of length $n + \ell$ for user-specified ℓ .

3. OWF from PRG (backward direction, PA#1b):

Show that $f(s) = G(s)$ is a OWF. Provide a brief written argument (code comment or README) and a demonstration that an adversary given $G(s)$ cannot recover s efficiently.

4. Statistical test suite:

Run your PRG output through at least three NIST SP 800-22 tests: frequency (monobit), runs, and serial. Report pass/fail and p -values.

5. Interface:

Your PRG must expose `seed(s)`, `next_bits(n)` so that PA#2 can consume it as a black box.

PA #1 — Interactive Demo Deliverable

Demo: Live PRG output viewer. In the web app, PA#1's demo panel shows:

- A hex input field for the seed s (e.g., 16 bytes).
- A slider for output length ℓ (8–256 bytes).
- Live output: as the user types or moves the slider, $G(s)$ updates instantly.
- A “Randomness test” button that runs the frequency and runs tests and displays pass/fail with a bar showing the bit ratio ($\approx 50\%$ ones expected).

Toy parameters: 64-bit seed, DLP group of order $\approx 2^{30}$ or AES-128. Must run in under 1 second.

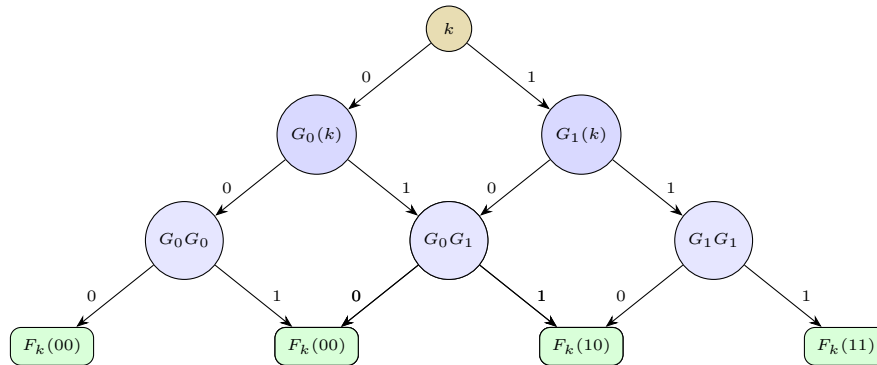
2.7 PA #2 — Pseudorandom Functions via GGM Tree

A PRF $F_k : \{0,1\}^n \rightarrow \{0,1\}^n$ maps an input x to an output that is computationally indistinguishable from a truly random function, when the key k is secret.

The GGM Tree Construction

Given a length-doubling PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, write $G(s) = G_0(s) \| G_1(s)$ (left and right halves). Define:

$$F_k(b_1 b_2 \dots b_n) = G_{b_n}(G_{b_{n-1}}(\dots G_{b_1}(k) \dots))$$



Depth-2 GGM tree for $n = 2$ input bits

PRF from PRG

Inputs: Key $k \in \{0, 1\}^n$, query $x = b_1 \dots b_n \in \{0, 1\}^n$

Algorithm:

1. Set $s_0 \leftarrow k$
2. For $i = 1$ to n : set $s_i \leftarrow G_{b_i}(s_{i-1})$
3. Output s_n

Cost: n evaluations of G ; tree has 2^n leaves total but only one root-to-leaf path is computed per query.

Plugging in AES: For practical purposes, AES-128 is already a highly efficient PRP/PRF. You may use $F_k(x) = \text{AES}_k(x)$ directly, bypassing the GGM construction.

Security Claim

If G is a secure PRG, then the GGM construction yields a secure PRF. Concretely: any PPT adversary making q queries to F_k cannot distinguish it from a random oracle with advantage greater than $\text{negl}(n)$.

PA #2 — What You Must Implement

1. **GGM PRF from your PA#1 PRG** (forward direction, PA#2a): Implement the depth- n GGM binary tree. Key $k \in \{0, 1\}^n$, input $x \in \{0, 1\}^n$; follow the root-to-leaf path defined by the bits of x , applying G_0 or G_1 at each level. Output the leaf value $F_k(x)$.
2. **PRG from PRF** (backward direction, PA#2b): Implement $G(s) = F_s(0^n) \| F_s(1^n)$ as a length-doubling PRG. Show this produces pseudorandom output by running it through the same statistical test suite as PA#1.
3. **AES plug-in**: Implement an alternative PRF using AES-128 directly: $F_k(x) = \text{AES}_k(x)$ (your own AES or the OS primitive as the one allowed exception). Show that substituting AES for your GGM PRF produces functionally identical results downstream.
4. **Distinguishing game demo**: Write a test that queries your PRF on $q = 100$ random inputs and a truly random function on the same inputs, and confirms no statistical difference (supporting PRF security empirically).
5. **Interface**: Expose $F(k, x)$ so that PA#3, PA#4, and PA#5 can use it as a drop-in.

PA #2 — Interactive Demo Deliverable

Demo: GGM tree visualiser. In the web app:

- Input: key k (hex) and a query x (bit string of length $n \leq 8$).
- Display: the GGM binary tree up to depth n , with each node showing its value in hex. The path taken for query x is highlighted in blue; inactive nodes are greyed out.
- The leaf value is labelled “ $F_k(x) =$ ” and shown in a prominent output box.
- Changing any input bit of x re-highlights the path instantly, showing how a 1-bit change in the query produces an uncorrelated output.

Toy parameters: $n = 4$ – 8 bits (tree depth), so the diagram stays readable. AES or your PRG as leaves.

2.8 PA #3 — CPA-Secure Symmetric Encryption**Intuition**

Chosen-Plaintext Attack (CPA): The adversary can encrypt arbitrary messages of their choice (e.g., by sending them to an encryption oracle). CPA security demands that the adversary cannot learn anything about which of two equal-length messages was encrypted, even after seeing polynomially many encryptions.

Standard CPA-Secure Construction

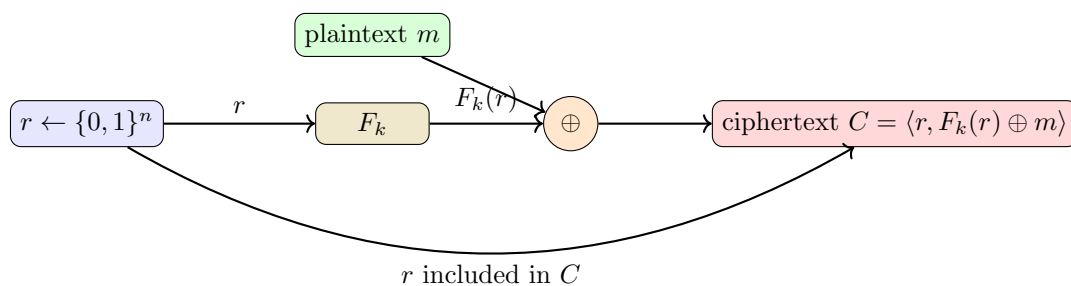
Enc-then-PRF Construction

Key: $k \in \{0, 1\}^n$ **Encryption** of $m \in \{0, 1\}^n$:

1. Sample $r \leftarrow \{0, 1\}^n$ uniformly at random (fresh each encryption).
2. Output $C = \langle r, F_k(r) \oplus m \rangle$

Decryption of $C = \langle r, c \rangle$:

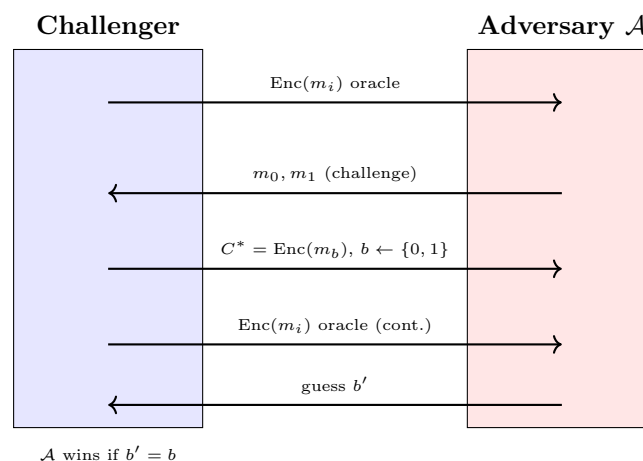
1. Compute $m = F_k(r) \oplus c$



Security Claim

The scheme above is CPA-secure if F_k is a secure PRF. The key is that r is *fresh and random* for each encryption; reusing r breaks security catastrophically.

CPA Security Game



PA #3 — What You Must Implement

1. **CPA-secure encryption scheme:** Implement $\text{Enc}(k, m)$ and $\text{Dec}(k, c)$ using the construction $C = \langle r, F_k(r) \oplus m \rangle$ where r is freshly sampled each call. Your F_k must be your own PRF from PA#2 (not a library).

2. **Multi-block support:** Extend to messages longer than one block by applying the PRF to $r, r+1, r+2, \dots$ (counter-based extension). Correctly handle messages whose length is not a multiple of the block size (padding).
3. **CPA game simulation:** Implement the IND-CPA game. Run it with a dummy adversary that queries the encryption oracle 50 times and then attempts to distinguish — confirm their advantage is ≈ 0 .
4. **Broken variant (demonstrate the attack):** Implement a version that reuses r (i.e., deterministic encryption). Show that this version is *broken* by an adversary that queries two equal messages and detects the identical ciphertexts.
5. **Interface:** Expose $\text{Enc}(k, m) \rightarrow (r, c)$ and $\text{Dec}(k, r, c) \rightarrow m$ for use in PA#6.

PA #3 — Interactive Demo Deliverable

Demo: Play the IND-CPA game. The student acts as the adversary:

- Step 1: Student types two messages m_0 and m_1 of equal length.
- Step 2: Click “Encrypt.” The challenger picks a random bit b and shows $C^* = \text{Enc}_k(m_b)$.
- Step 3: Student guesses b . The app reveals whether the guess was correct and updates a running advantage counter over multiple rounds (should converge to ≈ 0).
- Bonus: a “Reuse nonce” toggle breaks security — with nonce reuse, a correct guess is trivially forced and the advantage counter jumps to 1.

Run 20 rounds; confirm advantage ≤ 0.1 in secure mode and $= 1.0$ in broken mode.

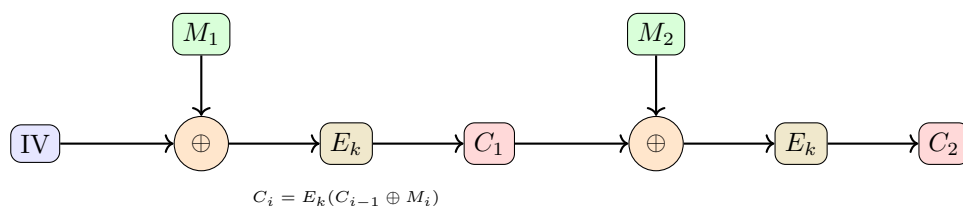
2.9 PA #4 — Modes of Operation

Intuition

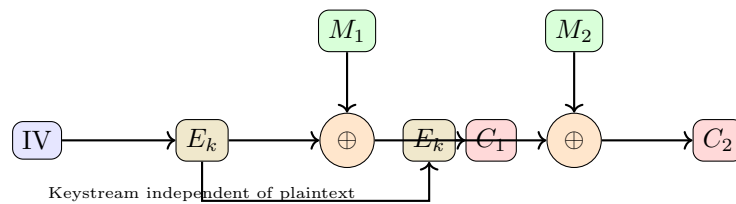
Block ciphers operate on fixed-size blocks (e.g., 128-bit for AES). Real messages are longer. *Modes of operation* specify how to chain block cipher evaluations to handle arbitrary-length messages securely. Different modes offer different trade-offs in parallelism, error propagation, and security properties.

You must support:

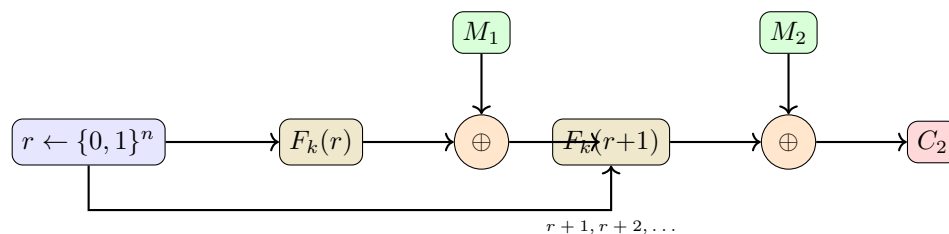
Mode 1: CBC (Cipher Block Chaining)



Properties: Sequential encryption (cannot parallelize encryption), parallel decryption, one-block error propagation. Requires random IV.

Mode 2: OFB (Output Feedback)

Properties: Pre-computable keystream; decryption identical to encryption. Keystream never reuses state. Bit flips in ciphertext produce bit flips in plaintext (no error propagation).

Mode 3: Randomized CTR (Counter Mode)

Properties: Fully parallelizable (both encryption and decryption), turns block cipher into stream cipher. The nonce r must be unique per key.

Comparison Table

Mode	Parallel Enc.	Parallel Dec.	Random Access	Error Prop.	IV Reuse
CBC		✓		2 blocks	Fatal
OFB				None	Fatal
CTR	✓	✓	✓	None	Fatal

PA #4 — What You Must Implement

- CBC mode:** Implement `CBC.Enc(k, IV, M)` and `CBC.Dec(k, IV, C)` for arbitrary-length messages. The IV must be randomly generated per encryption call. Use your own PRF/PRP from PA#2 (or your own AES) as the block cipher E_k .
- OFB mode:** Implement `OFB.Enc(k, IV, M)` and `OFB.Dec(k, IV, C)`. Confirm that encryption and decryption are identical operations. Demonstrate that the keystream can be pre-computed before the plaintext is known.
- Randomized CTR mode:** Implement `CTR.Enc(k, M)` (samples random nonce r internally) and `CTR.Dec(k, r, C)`. Demonstrate parallel block computation.
- Mode selector interface:** Expose a unified API `Encrypt(mode, k, M)` and `Decrypt(mode, k, C)` where `mode` $\in$ {CBC, OFB, CTR}. Internally routes to the correct implementation.
- Attack demos:** (a) Show CBC IV-reuse attack: encrypt two different messages with the same IV and demonstrate that block i leaks if $M_i = M'_i$. (b) Show OFB keystream-reuse attack: encrypt two messages with the same IV and XOR the ciphertexts to recover plaintext XOR.

6. **Correctness tests:** For each mode, verify $\text{Dec}(k, \text{Enc}(k, M)) = M$ on at least three message lengths: shorter than one block, exactly one block, and spanning multiple blocks.

PA #4 — Interactive Demo Deliverable

Demo: Block cipher mode animator. For a 3-block message:

- Three mode tabs: CBC, OFB, CTR. Switching tabs re-runs the animation.
- Each block is a coloured box. Arrows animate in sequence showing XOR operations and block cipher calls, with each intermediate value shown in hex.
- “Flip bit” button: student clicks any ciphertext block; the app re-decrypts and highlights which plaintext blocks are corrupted (CBC: 2 blocks; OFB: only the same block; CTR: only the same block).
- “Reuse IV” toggle: re-encrypts two messages with the same IV in CBC mode and highlights the matching ciphertext blocks in red.

Flip a ciphertext bit in each mode and verify correct error propagation pattern.

2.10 PA #5 — Message Authentication Codes (MACs)

Intuition

The need for integrity: Encryption provides *confidentiality* but not *integrity*. An adversary can flip bits in a ciphertext and the decryption will produce a modified (but valid-looking) plaintext. A MAC allows a receiver to verify that a message was not tampered with, without requiring a public key.

MAC Definition

A MAC scheme $(\text{Mac}, \text{Vrfy})$ with key space $\mathcal{K}$, message space $\mathcal{M}$, and tag space $\mathcal{T}$ satisfies:

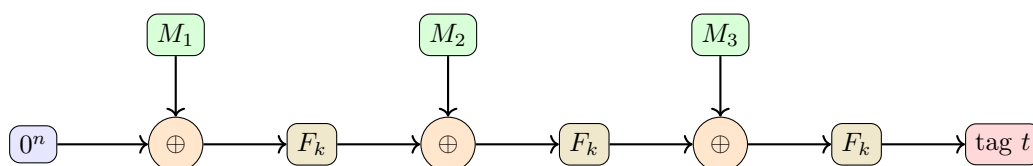
- $\text{Mac}_k(m) \rightarrow t$: produce tag t for message m under key k .
- $\text{Vrfy}_k(m, t) \rightarrow \{0, 1\}$: verify that t is a valid tag for m .

Security (EUF-CMA): No PPT adversary making polynomially many $(m_i, \text{Mac}_k(m_i))$ queries can forge a valid tag on a *new* message.

Three constructions to implement:

Construction 1: PRF-MAC (Fixed-length) For messages $m \in \{0, 1\}^n$: set $t = F_k(m)$.

Construction 2: CBC-MAC (Variable-length)



Construction 3: HMAC Using a hash function H : $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m))$.

PA #5 — What You Must Implement

1. **PRF-MAC (fixed-length)**: Implement $\text{Mac}(k, m) = F_k(m)$ and $\text{Vrfy}(k, m, t)$. The PRF F_k must be your own from PA#2. This handles messages of exactly one block length n .
2. **CBC-MAC (variable-length)**: Extend to arbitrary-length messages using the CBC-MAC construction (chain F_k over message blocks, output the final chaining value as the tag). Implement both Mac and Vrfy .
3. **HMAC (forward pointer)**: PA#5 introduces the HMAC formula for completeness, but the full implementation of HMAC belongs in **PA#10**. In PA#5, you may leave a clearly labelled stub function $\text{hmac}(k, m)$ that raises `NotImplemented` until PA#10 is complete. *Do not use a library HMAC here or in PA#10.*
4. **MAC $\Rightarrow$ PRF (backward direction)**: Demonstrate that your PRF-MAC, when queried on uniformly random inputs, passes the same PRF distinguishing test from PA#2. This concretely witnesses $\text{MAC} \Rightarrow \text{PRF}$.
5. **EUFCMA forgery demo**: Implement the EUFCMA game. Show that a naive adversary (who has seen up to 50 (m_i, t_i) pairs) cannot produce a valid tag on a new message. Separately, demonstrate a *length-extension attack* on a naive single-hash MAC ($t = H(k \| m)$) to motivate the HMAC double-hash structure.
6. **Interface**: Expose $\text{Mac}(k, m) \rightarrow t$ and $\text{Vrfy}(k, m, t) \rightarrow \text{bool}$ for use in PA#6.

PA #5 — Interactive Demo Deliverable

Demo: MAC forge attempt. The student acts as the EUFCMA adversary:

- The app shows a running list of up to 50 signed messages (m_i, t_i) generated by a hidden key k .
- Student types a new message m^* (not in the list) and a tag t^* .
- Click “Submit forgery.” The app verifies and shows “Forgery accepted” or “Forgery rejected.”
- A counter tracks forgery attempts and successes. Graders expect 0 successes in ≥ 20 attempts.
- Separate tab: “Length-extension demo” — student types a suffix m' , app shows the extended tag on $m \| \text{pad} \| m'$ being computed from t alone (without k), demonstrating the naive $H(k \| m)$ vulnerability.

2.11 PA #6 — CCA-Secure Symmetric Encryption

Intuition

Why CPA is not enough: CPA-secure schemes are *malleable*—an adversary can modify a ciphertext in a predictable way to alter the plaintext without knowing the key. CCA (Chosen-Ciphertext Attack) security eliminates this by requiring that even an adversary with a decryption oracle cannot break security. The **Encrypt-then-MAC** paradigm achieves this.

Encrypt-then-MAC Construction

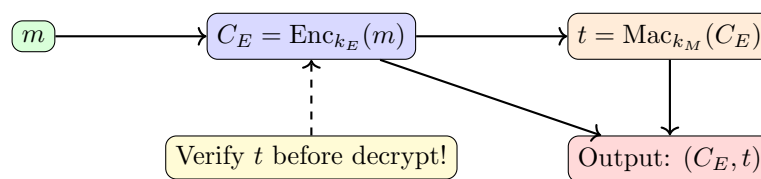
Keys: Independent encryption key k_E and MAC key k_M .

Encryption of m :

1. $C_E \leftarrow \text{Enc}_{k_E}(m)$ (CPA-secure, e.g., PA#3)
2. $t \leftarrow \text{Mac}_{k_M}(C_E)$ (EUF-CMA secure, e.g., PA#5)
3. Output (C_E, t)

Decryption of (C_E, t) :

1. If $\text{Vrfy}_{k_M}(C_E, t) = 0$: output $\perp$ (reject tampered ciphertext)
2. Else: output $\text{Dec}_{k_E}(C_E)$



MAC check prevents CCA oracle exploitation

Security Claim

If Enc is CPA-secure and Mac is EUF-CMA secure, then Encrypt-then-MAC is **CCA2-secure**. The MAC verification step means any adversary query to the decryption oracle on a modified ciphertext is rejected, nullifying the adaptive oracle.

PA #6 — What You Must Implement

1. **Encrypt-then-MAC:** Implement $\text{CCA_Enc}(k_E, k_M, m)$ and $\text{CCA_Dec}(k_E, k_M, c, t)$. Internally, CCA_Enc calls your PA#3 Enc and your PA#5 Mac. CCA_Dec must call *Vrfy* before calling *Dec* — rejecting with $\perp$ on MAC failure.
2. **Key separation:** Use independently sampled keys k_E and k_M (never the same key for both). Demonstrate that reusing a single key for both roles creates exploitable correlations.
3. **CCA2 game simulation:** Implement the IND-CCA2 game. Your adversary gets both an encryption oracle and a decryption oracle (which rejects the challenge ciphertext). Show that your scheme's adversary advantage is ≈ 0 .
4. **Malleability attack on CPA-only:** Show that your PA#3 CPA-secure scheme (without the MAC) is *malleable*: given $C = \langle r, F_k(r) \oplus m \rangle$, an adversary can flip bit i of the ciphertext to produce a ciphertext for m with bit i flipped, without knowing k or m . Contrast this with your CCA-secure scheme where the same attack is detected and rejected.
5. **Interface:** Expose $\text{CCA_Enc}(k_E, k_M, m) \rightarrow (c, t)$ and $\text{CCA_Dec}(k_E, k_M, c, t) \rightarrow m$ or $\perp$.

PA #6 — Interactive Demo Deliverable

Demo: Malleability attack panel.

- Left side (CPA-only): student is shown a ciphertext $C = \langle r, F_k(r) \oplus m \rangle$. A bit-flip

tool lets them click any bit. The app decrypts the modified ciphertext and shows the corrupted plaintext — demonstrating malleability.

- Right side (CCA / Encrypt-then-MAC): the same bit-flip attempt is shown, but the MAC verification step fires and returns $\perp$. The plaintext never decrypts.
- Both sides update live as the student flips different bits.

Flip a bit on the left; confirm modified plaintext appears. Flip the same bit on the right; confirm $\perp$ is returned.

3 Part II: Hashing and Data Integrity

Standalone Concept: The Birthday Bound (Theory)

Standalone Concept: Birthday Bound — Security Floor of Every Hash Function

The Birthday Bound is a **standalone, information-theoretic result**: it applies to *any* hash function regardless of how it is constructed, and it defines a hard security ceiling that no engineering can overcome.

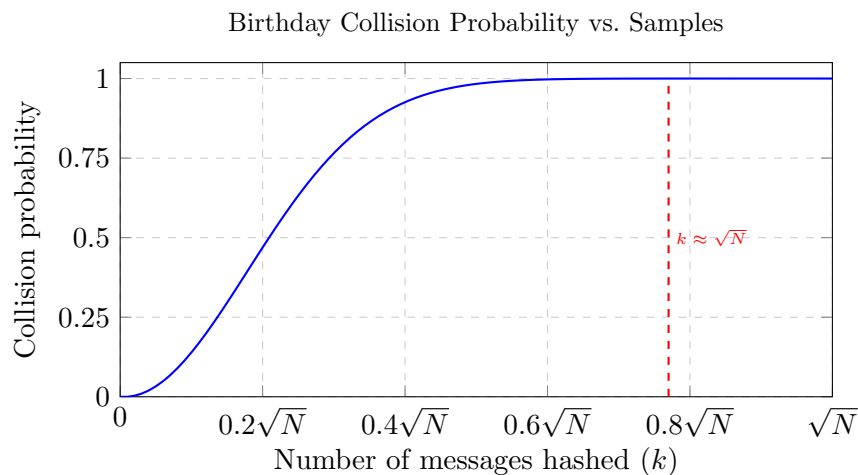
The Birthday Problem: In a group of k people, the probability that two share a birthday exceeds 50% when $k \approx \sqrt{365} \approx 23$. The intuition is that we are counting *pairs*, not individuals: k people form $\binom{k}{2} \approx k^2/2$ pairs.

Cryptographic consequence: For a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$:

- By the pigeonhole principle, collisions *must* exist (domain $>$ range).
- Finding a collision requires only $O(2^{n/2})$ random hash evaluations in expectation.
- This is the **absolute lower bound on hash function strength**: no hash with n -bit output can require more than $O(2^{n/2})$ work to collide.

Practical impact: MD5 ($n = 128$): $\approx 2^{64}$ ops to find collision (broken 2005). SHA-1 ($n = 160$): $\approx 2^{80}$ ops (broken 2017). SHA-256 ($n = 256$): $\approx 2^{128}$ ops (currently secure). This is why modern hashes use at least 256-bit output.

PA#9 implements this attack concretely — see Section 3.3. You will run the algorithm on your own hash functions and empirically confirm the $O(2^{n/2})$ prediction.



3.1 PA #7 — Merkle-Damgård Transform

Intuition

We want hash functions that accept inputs of *any* length but produce fixed-length outputs. The Merkle-Damgård transform is the **domain extension** technique: given a fixed-length compression function $h : \{0, 1\}^{n+b} \rightarrow \{0, 1\}^n$, it constructs a collision-resistant hash for arbitrary-length inputs. **Key theorem:** Any collision in the output of the MD

transform implies a collision in h .

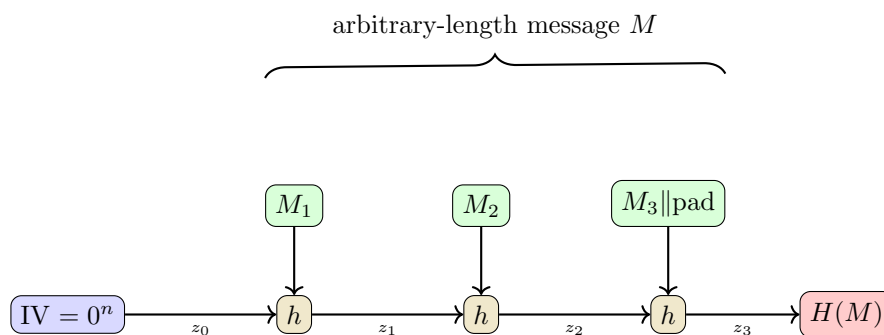
Merkle-Damgrd Transform

Input: Message M of any length; fixed $IV = 0^n$

Preprocessing: Pad M to a multiple of b bits using *MD-strengthening* padding:
 $M \parallel 1 \parallel 0^* \parallel \langle |M| \rangle$ (append 1-bit, then zeros, then 64-bit length field)

Algorithm:

1. Parse padded message as blocks $M_1, M_2, \dots, M_\ell \in \{0, 1\}^b$
2. $z_0 \leftarrow IV$
3. For $i = 1$ to ℓ : $z_i \leftarrow h(z_{i-1} \parallel M_i)$
4. Output z_ℓ



Security Claim

If h is a collision-resistant compression function, then the MD transform H is collision-resistant. **Proof sketch:** If adversary finds $M \neq M'$ with $H(M) = H(M')$, trace back through the chain to find a collision in h (possible because h 's inputs differ whenever the chaining values differ).

PA #7 — What You Must Implement

1. **Merkle-Damgrd framework:** Implement a generic `MerkleDamgard(compress, IV, block.size)` class/function that accepts *any* compression function $h : \{0, 1\}^{n+b} \rightarrow \{0, 1\}^n$ as a parameter and produces a hash function for arbitrary-length inputs.
2. **MD-strengthening padding:** Implement the padding correctly: append a 1-bit, then enough 0-bits, then a 64-bit big-endian encoding of the original message length, so the padded message is a multiple of b bits.
3. **Dummy compression plug-in:** For testing PA#7 in isolation, supply a toy compression function (e.g., XOR-based) to verify the MD framework produces correct-length outputs and handles boundary cases (empty message, exactly-one-block message, multi-block message).
4. **Collision propagation demo:** Show the reduction: given two inputs that collide under your toy compression function, demonstrate that they also collide under the full MD hash. This concretely illustrates why security of H reduces to security of h .
5. **Interface:** Expose `hash(message, compression_fn) -> digest` so that PA#8 can plug in the DLP compression function directly.

PA #7 — Interactive Demo Deliverable

Demo: Merkle-Damgård chain viewer.

- Student types an arbitrary message (text or hex).
- The app splits it into blocks, applies MD-strengthening padding, and displays each block as a labelled box.
- An animated chain shows $z_0 \rightarrow h(z_0, M_1) \rightarrow h(z_1, M_2) \rightarrow \dots$ with each chaining value shown in hex.
- Editing any block re-computes the chain from that block onwards (showing avalanche effect).

Toy parameters: Use your XOR-based toy compression function from PA#7's test suite for speed. Block size 8 bytes, output 4 bytes.

3.2 PA #8 — DLP-Based Collision-Resistant Hash Function

Intuition

The Merkle-Damgård transform provides *structure* but not security—security comes from the compression function. Here we instantiate h using the **Discrete Logarithm Problem**, giving a provably collision-resistant compression function under the DL assumption. This is a purely number-theoretic construction with a clean security proof.

Setup

Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of prime order q where DLP is hard (e.g., a safe-prime subgroup of $\mathbb{Z}_p^*$, or an elliptic curve group). Let $g, h \in \mathbb{G}$ with $h = g^\alpha$ for unknown α (i.e., $\log_g h$ is unknown).

DLP Compression Function

DLP-based Compression Function

Define $\text{Compress} : \mathbb{Z}_q \times \mathbb{Z}_q \rightarrow \mathbb{G}$:

$$h(x, y) = g^x \cdot h^y = g^{x+\alpha y} \pmod{p}$$

Equivalently written as $h(x, y) = g^{x+y}$ if we set $h = g$ (simplest case):

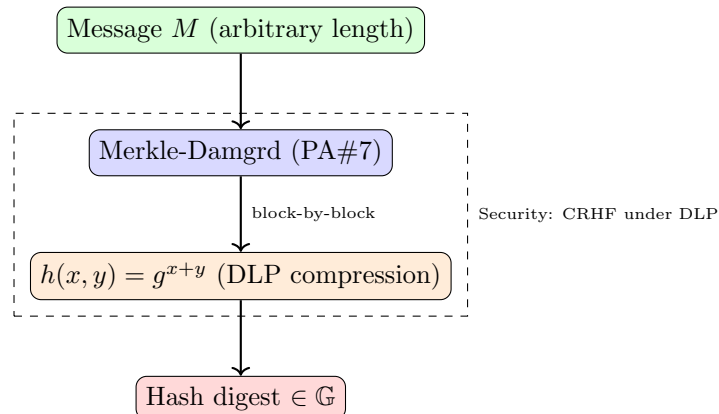
$$h(x, y) = g^{x+y} \pmod{p}$$

Collision Resistance Proof Sketch: Suppose adversary finds $(x, y) \neq (x', y')$ with $h(x, y) = h(x', y')$, i.e.:

$$g^x \cdot \hat{h}^y = g^{x'} \cdot \hat{h}^{y'} \pmod{p}$$

Then $g^{x-x'} = \hat{h}^{y'-y}$, so $\log_g \hat{h} = \frac{x-x'}{y'-y} \pmod{q}$. This would solve DLP, contradicting our assumption!

Full Pipeline



PA #8 — What You Must Implement

1. **Group setup:** Select and implement a prime-order group where DLP is hard. Acceptable choices: (a) a safe-prime subgroup of $\mathbb{Z}_p^*$ with $p = 2q + 1$, q prime, group order q ; (b) an elliptic curve group (e.g., Curve25519 parameters — but implement the arithmetic yourself, no library). Generate g and $\hat{h} = g^\alpha$ for a randomly chosen α that is then *discarded* (nobody should know α).
2. **DLP compression function:** Implement $h(x, y) = g^x \cdot \hat{h}^y \bmod p$ (or the elliptic curve analogue) as a function mapping two $\mathbb{Z}_q$ inputs to one group element. This is your compression function for PA#7.
3. **Full CRHF:** Plug your DLP compression function into the PA#7 Merkle-Damgrd framework to produce a complete collision-resistant hash function `DLP_Hash(message) -> group_element`.
4. **Collision resistance demo:** Show that you cannot find a collision without solving DLP. Specifically: (a) demonstrate that random pairs $(x, y) \neq (x', y')$ with $h(x, y) = h(x', y')$ would require computing $\log_g \hat{h}$; (b) write a brute-force collision finder for a tiny parameter ($q \approx 2^{16}$) and show it requires enumerating $O(2^8) = O(\sqrt{q})$ values (birthday bound).
5. **Integration test:** Hash at least five messages of different lengths through `DLP_Hash` and confirm distinct inputs produce distinct digests.
6. **Forward pointer to PA#10:** Your `DLP_Hash` is the hash function that PA#10 will wrap in HMAC. Ensure the interface matches: `DLP_Hash(message) -> bytes` with configurable output length.

PA #8 — Interactive Demo Deliverable

Demo: DLP hash live.

- Student types a message; the app computes `DLP_Hash(message)` and displays the group element as a hex string.
- A “Collision hunt” button runs the birthday attack (toy $n = 16$ -bit output) in the background, updating a counter of hashes evaluated, and highlights the two colliding inputs when found.
- A progress bar shows how close the count is to $2^{n/2} = 256$.

Toy parameters: $q \approx 2^{16}$, truncated to 16-bit output for the collision demo. Full-size hash shown separately.

3.3 PA #9 — Birthday Attack (Collision Finding)

Intuition

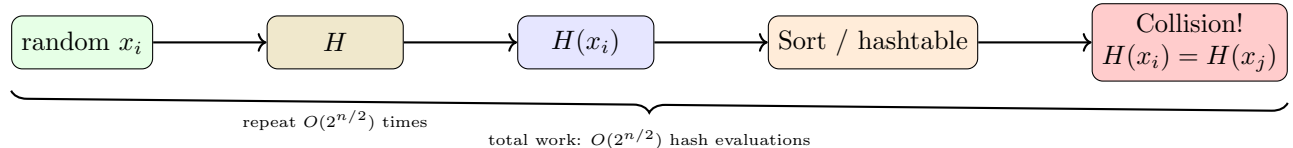
Theory meets implementation: The Birthday Attack is not merely a theoretical bound—it is an *executable algorithm*. PA#9 asks you to implement it, run it on your own hash functions with small parameters, and empirically confirm the $O(2^{n/2})$ behaviour. This grounds the abstract security analysis in concrete experimental evidence, and demonstrates that your CRHF from PA#8 is *exactly as hard* as the birthday bound predicts—no better, no worse.

The Algorithm in Detail

Two standard approaches:

Approach 1: Naive birthday (sort-based) Hash $k = \lceil 1.2 \cdot 2^{n/2} \rceil$ random inputs, sort by output, scan for duplicates. Time $O(k \log k)$, space $O(k)$.

Approach 2: Floyd’s cycle detection (space-efficient) Treat the hash as a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ (set $f(x) = H(x)$ truncated to n bits). Use tortoise-and-hare to find a cycle, then extract a collision. Time $O(2^{n/2})$, space $O(1)$.



Security Claim

For any hash with n -bit output: (a) no algorithm can find a collision in fewer than $\Omega(2^{n/2})$ queries (lower bound); (b) the birthday algorithm finds one in $O(2^{n/2})$ queries (upper bound). The bound is therefore *tight*. Increasing output length by 1 bit doubles the cost of finding a collision.

PA #9 — What You Must Implement

- Naive birthday algorithm:** Implement `birthday_attack(hash_fn, n, num_trials)` that hashes random inputs, stores outputs in a dictionary, and returns the first collision pair (x, x') with $x \neq x'$ and $H(x) = H(x')$. n is the output bit length; `hash_fn` is any callable.
- Floyd’s cycle-finding attack:** Implement the space-efficient variant using tortoise-and-hare on $f(x) = H(x)$ (treating the n -bit output as the next input). Return the collision pair without storing all intermediate hashes.
- Attack your own toy hash:** Design a deliberately weak hash function with $n = 8, 12, 16$ output bits. Run both algorithms on it. Plot the number of evaluations needed against $2^{n/2}$ for each n value and confirm the empirical count matches the theoretical prediction.

4. **Attack truncated DLP hash:** Take your PA#8 DLP hash and truncate its output to $n = 16$ bits. Run the birthday attack. Report: (a) how many evaluations were needed; (b) the ratio evaluations/ $2^{n/2}$; (c) the two colliding inputs. This concretely shows that even a provably-secure hash is broken at the birthday bound if its output is too short.
5. **Empirical birthday curve:** Run 100 independent trials for each of $n \in \{8, 10, 12, 14, 16\}$ bits. For each n , plot the distribution of the number of queries until first collision. Overlay the theoretical curve $1 - e^{-k(k-1)/2^{n+1}}$. Confirm the match.
6. **MD5/SHA-1 context:** Compute $2^{n/2}$ for $n = 128$ (MD5) and $n = 160$ (SHA-1). Express the result in terms of modern CPU speed (e.g., if a CPU hashes 10^9 values/sec, how many seconds/years does the attack take?). This contextualises why MD5 is broken and SHA-1 is deprecated.

PA #9 — Interactive Demo Deliverable

Demo: Live birthday attack.

- A slider lets the student pick output bit-length $n \in \{8, 10, 12, 14, 16\}$.
- Click “Run attack.” A counter increments live as hashes are computed. When a collision is found, the two inputs and their shared hash are displayed.
- A live chart plots “hashes computed” vs “collision probability” and overlays the theoretical curve $1 - e^{-k^2/2^n}$.
- The expected collision point $2^{n/2}$ is shown as a vertical marker; the student sees the empirical collision landing near it.

Run for $n = 12$; confirm collision found near $2^6 = 64$ evaluations on average.

3.4 PA #10 — HMAC and HMAC-Based CCA-Secure Encryption

Intuition

The HMAC bridge: PA#10 is the assignment where the two worlds of the course—symmetric cryptography (Part I) and hashing (Part II)—meet. HMAC takes your hash function (PA#8) and produces a MAC (a Part I primitive). The security proof of HMAC uses the PRF security of the compression function, not collision resistance. This is what earns HMAC its place in the Minicrypt clique: it is the concrete witness of the CRHF $\leftrightarrow$ MAC equivalence.

Then, armed with a hash-based MAC, we rebuild CCA-secure encryption using Encrypt-then-HMAC, closing the loop back to PA#6 but now with a hash-derived authentication tag instead of a PRF-derived one.

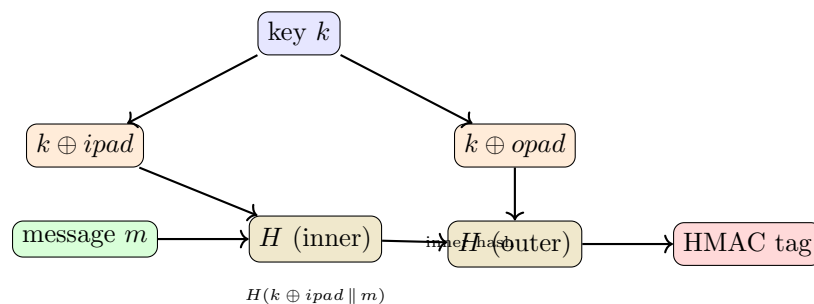
HMAC Construction

HMAC Definition

Let $H : \{0,1\}^* \rightarrow \{0,1\}^n$ be a hash function with block size b bytes. For key k (padded/hashed to b bytes) and message m :

$$\text{HMAC}_k(m) = H(\underbrace{(k \oplus \text{opad})}_{\text{outer key}} \parallel \underbrace{H(\underbrace{(k \oplus \text{ipad})}_{\text{inner key}} \parallel m)}_{\text{inner hash}})$$

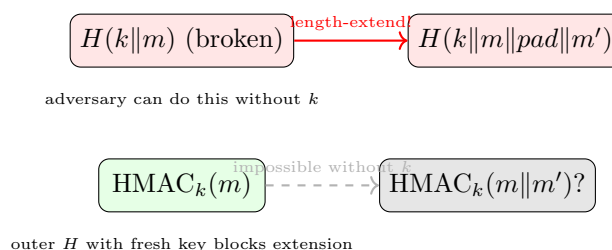
where $\text{ipad} = 0x36^b$ and $\text{opad} = 0x5C^b$ are fixed constants.



Why Two Hashes? The Length-Extension Attack

A naive MAC $t = H(k||m)$ is broken by a **length-extension attack**: given $H(k||m)$, an adversary can compute $H(k||m||pad||m')$ for any suffix m' without knowing k , because the Merkle-Damgrd state after processing $k||m$ is exactly $H(k||m)$.

HMAC defeats this by using the inner hash output as a fixed-length input to a *fresh* outer hash invocation keyed with $k \oplus \text{opad}$. The adversary cannot continue the inner hash without restarting from the keyed IV.



HMAC-Based CCA-Secure Encryption

Once HMAC is implemented, we rebuild the Encrypt-then-MAC paradigm from PA#6, this time using HMAC as the authentication layer. This is the practical standard (TLS 1.2 uses exactly this pattern):

$$C = \text{Enc}_{k_E}(m), \quad t = \text{HMAC}_{k_M}(C), \quad \text{send } (C, t)$$

$$\text{Verify: } \text{HMAC}_{k_M}(C) \stackrel{?}{=} t \text{ before decrypting}$$

Security Claim

HMAC is EUF-CMA secure if the compression function h of H is a PRF. This is weaker than requiring collision resistance of H itself — HMAC remains secure even against adversaries who can find collisions in H (as happened to MD5), as long as h retains PRF security. This is why HMAC-MD5 was used safely in TLS long after MD5 collisions were found.

PA #10 — What You Must Implement

1. **HMAC over your PA#8 hash:** Implement $\text{HMAC}(k, m)$ using your own `DLP.Hash` from PA#8 as the underlying H . Implement key padding (if $|k| > b$, hash it first; if $|k| < b$, zero-pad to b bytes). Implement $\text{HMAC_Verify}(k, m, t) \rightarrow \text{bool}$ using a constant-time comparison to prevent timing attacks.
2. **CRHF $\Rightarrow$ MAC (forward, bidirectional):** Demonstrate that your HMAC is a secure MAC by running the EUF-CMA game: after 50 queries to the HMAC oracle, confirm the adversary cannot forge a tag on a new message.
3. **MAC $\Rightarrow$ CRHF (backward, bidirectional):** Demonstrate the reverse direction. Construct a new compression function $h'(cv, block) = \text{HMAC}_k(cv || block)$ for a fixed public key k . Plug h' into your PA#7 Merkle-Damgrd framework to produce a new hash function `MAC.Hash`. Show that finding a collision in `MAC.Hash` would require forging an HMAC tag.
4. **Length-extension attack demo:** Show concretely that the naive MAC $t = H(k || m)$ (using your PA#8 hash directly, *without* the HMAC structure) is broken by a length-extension attack: given (m, t) , construct a valid $(m || \text{pad} || m', t')$ for any chosen m' . Then show the same attack fails on HMAC.
5. **Encrypt-then-HMAC (CCA-secure encryption):** Implement $\text{EtH_Enc}(kE, kM, m)$ using your PA#3 CPA-secure encryption and your new HMAC: encrypt first, then HMAC the ciphertext. Implement $\text{EtH_Dec}(kE, kM, c, t)$: verify HMAC before decrypting; return $\perp$ on failure.
6. **CCA2 game for Encrypt-then-HMAC:** Run the IND-CCA2 game. Confirm the scheme correctly rejects modified ciphertexts and achieves negligible adversary advantage. Compare performance (tag size, computation cost) against the PRF-MAC-based scheme from PA#6.
7. **Constant-time comparison:** Implement `secure_compare(t1, t2)` that compares two tags in constant time (XOR all bytes, check result is zero). Demonstrate that a naive early-exit comparison leaks the tag via a timing side channel (measure time difference for tags differing in early vs. late bytes).
8. **Interface:** Expose $\text{HMAC}(k, m) \rightarrow \text{tag}$, $\text{HMAC_Verify}(k, m, t) \rightarrow \text{bool}$, $\text{EtH_Enc}(kE, kM, m) \rightarrow (c, t)$, and $\text{EtH_Dec}(kE, kM, c, t) \rightarrow m \text{ or } \perp$.

PA #10 — Interactive Demo Deliverable

Demo: Length-extension vs HMAC side-by-side.

- Left panel (broken $H(k || m)$): student is shown (m, t) . They type a suffix m' . The app computes a valid tag for $m || \text{pad} || m'$ without knowing k and shows “Forgery succeeded.”
- Right panel (HMAC): same attempt. The app shows “Forgery failed” — computing $\text{HMAC}_k(m || \text{pad} || m')$ requires k .
- A toggle switches the underlying hash between your PA#8 DLP hash and a place-

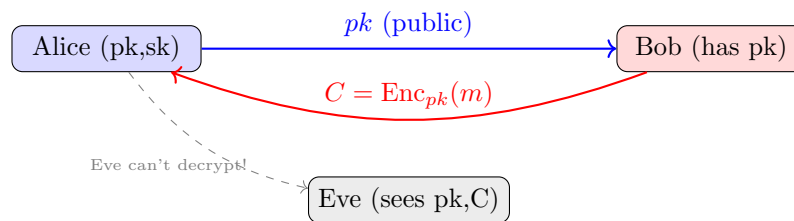
holder SHA-256 (for comparison), showing HMAC works with both.

Confirm length-extension succeeds on the left and fails on the right for any typed suffix.

4 Part III: Public-Key Cryptography (Cryptomania)

Intuition

The Key Distribution Problem: Symmetric cryptography requires Alice and Bob to share a secret key *before* communicating. How do they do this over an insecure channel? Public-key cryptography (PKC), powered by *trapdoor* one-way functions, solves this: everyone can encrypt using a public key, but only the holder of the private key (trapdoor) can decrypt. This jumps us from Minicrypt to **Cryptomania**.



4.1 PA #11 — Diffie-Hellman Key Exchange (SKE)

Intuition

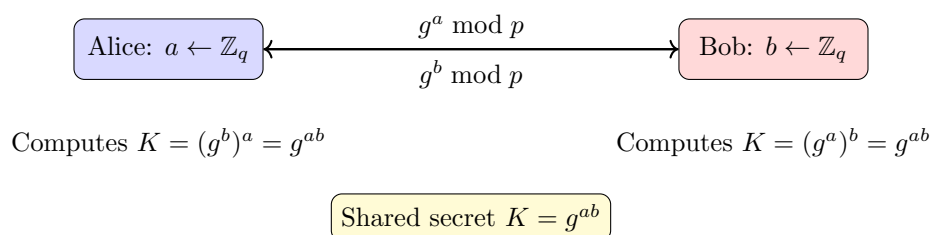
Two parties, Alice and Bob, wish to establish a shared secret over a completely public (eavesdropped) channel, without any prior shared secret. The DH protocol achieves this using the hardness of the **Computational Diffie-Hellman (CDH)** problem: given g^a and g^b , compute g^{ab} .

Diffie-Hellman Protocol

Public parameters: Prime p , generator g of $\mathbb{Z}_p^*$ (order q).

1. Alice samples $a \leftarrow \mathbb{Z}_q$, sends $A = g^a \bmod p$.
2. Bob samples $b \leftarrow \mathbb{Z}_q$, sends $B = g^b \bmod p$.
3. Alice computes $K = B^a = g^{ab}$. Bob computes $K = A^b = g^{ab}$.

Both now share $K = g^{ab}$, which Eve cannot compute (CDH assumption).



Security note: Basic DH is *not* authenticated—it is vulnerable to Man-in-the-Middle (MITM) attacks. Authentication requires digital signatures (PA#15).

PA #11 — What You Must Implement

1. **Group parameter generation:** Generate a safe prime $p = 2q + 1$ (using your PA#13 Miller-Rabin tester) and a generator g of the prime-order subgroup of $\mathbb{Z}_p^*$ of order q .
2. **DH key exchange protocol:** Implement both parties (Alice and Bob) as functions: `dh_alice_step1()` $\rightarrow$ (a, A) produces a private exponent and public value; `dh_bob_step1()` $\rightarrow$ (b, B) likewise; `dh_alice_step2(a, B)` $\rightarrow$ K and `dh_bob_step2(b, A)` $\rightarrow$ K compute the shared secret. Verify $K_A = K_B$.
3. **MITM attack demo:** Implement an active adversary Eve who intercepts A and B, substitutes her own values $A' = g^e$ and $B' = g^e$, and establishes separate shared secrets with Alice and Bob. Show she can read all traffic.
4. **CDH hardness:** For small parameters ($q \approx 2^{20}$), demonstrate that computing g^{ab} from g^a and g^b without knowing a or b requires a brute-force search. Report the time taken.

PA #11 — Interactive Demo Deliverable

Demo: Live Diffie-Hellman exchange.

- Two panels: Alice (left) and Bob (right). Each has an input for their private exponent a or b (or a “randomise” button).
- Click “Exchange.” The app animates: Alice sends g^a to Bob, Bob sends g^b to Alice.
- Both panels compute and display the shared secret $K = g^{ab}$, shown in green when they match.
- “Enable Eve” checkbox inserts a MITM: Eve intercepts, substitutes her own values, and both shared secrets change. Eve’s panel shows she now holds both secrets.

Toy parameters: $p \approx 2^{32}$ safe prime for instant computation. Values shown in hex.

4.2 PA #12 — Textbook RSA

Standalone Concept: Prime Factoring Assumption

The security of RSA rests on: given $N = p \cdot q$ for large primes p, q , it is computationally infeasible to recover p and q . (Best known algorithm: Number Field Sieve, runs in $e^{O((\ln N)^{1/3}(\ln \ln N)^{2/3})}$ time.)

RSA Key Generation and Cryptosystem

Key Generation:

1. Choose large primes p, q (use PA#13!); set $N = p \cdot q$.
2. Compute $\phi(N) = (p - 1)(q - 1)$.
3. Choose e with $\gcd(e, \phi(N)) = 1$ (commonly $e = 65537$).
4. Compute $d = e^{-1} \bmod \phi(N)$ via extended Euclidean algorithm.
5. Public key: (N, e) . Private key: (N, d) .

Encryption: $C = M^e \bmod N$

Decryption: $M = C^d \bmod N$ (since $C^d = M^{ed} = M^{1+k\phi(N)} = M$ by Euler’s theorem)

Warning: Textbook RSA is NOT Secure!

Textbook RSA is **deterministic**—encrypting the same plaintext always yields the same ciphertext. This immediately breaks CPA security. Never use raw RSA in practice; always apply padding. This assignment therefore requires **two** implementations: textbook RSA (to understand the raw structure) and PKCS#1 v1.5 padded RSA (to achieve practical security).

Required Extension: PKCS#1 v1.5 Padding

PKCS#1 v1.5 (RFC 2313) is the historically standard padding scheme that turns textbook RSA into a usable encryption primitive by injecting randomness and structure into the message before encryption.

PKCS#1 v1.5 Encryption Padding

Input: Message m of length $|m|$ bytes; RSA modulus N of k bytes.

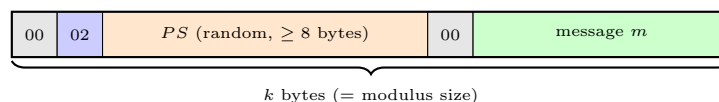
Require $|m| \leq k - 11$.

Padded plaintext EM :

$$EM = \underbrace{00}_{\text{leading zero}} \parallel \underbrace{02}_{\text{type}} \parallel \underbrace{PS}_{\text{random nonzero bytes, } \geq 8} \parallel \underbrace{00}_{\text{separator}} \parallel \underbrace{m}_{\text{message}}$$

- The 0x00 02 header signals encryption padding (vs. 0x00 01 for signatures).
- PS is at least 8 bytes of *cryptographically random, nonzero* bytes.
- Total length of EM equals k (the modulus byte length).

Encrypt: $C = EM^e \bmod N$ **Decrypt:** $EM = C^d \bmod N$, then strip padding.

**Bleichenbacher's Attack: Why Even PKCS#1 v1.5 Is Fragile**

In 1998, Daniel Bleichenbacher showed that a **padding oracle**—a service that reveals whether a ciphertext decrypts to a valid PKCS#1 v1.5 format—can be exploited to decrypt any RSA ciphertext with $\approx 2^{20}$ adaptive queries. This is an adaptive CCA2 attack.

Lesson: PKCS#1 v1.5 is CPA-secure but *not* CCA-secure. The modern replacement is **OAEP** (Optimal Asymmetric Encryption Padding), which achieves CCA2-security in the random oracle model. PA#17 addresses full CCA-secure PKC using the Sign-then-Encrypt paradigm.

PA #12 — What You Must Implement

1. **Key generation:** Implement `rsa.keygen(bits)` using your PA#13 Miller-Rabin tester to generate two $\lfloor \text{bits}/2 \rfloor$ -bit primes p, q . Compute $N = pq$, $\phi(N) = (p-1)(q-1)$, choose $e = 65537$, compute $d = e^{-1} \bmod \phi(N)$ using the extended Euclidean algorithm (implement this yourself).

2. **Textbook RSA:** Implement $\text{rsa_enc}(\text{pk}, m) = m^e \bmod N$ and $\text{rsa_dec}(\text{sk}, c) = c^d \bmod N$. Use fast modular exponentiation (square-and-multiply — implement this yourself, no library pow).
3. **PKCS#1 v1.5:** Implement $\text{pkcs15_enc}(\text{pk}, m)$ (pad m as $00\|02\|PS\|00\|m$ then apply textbook RSA) and $\text{pkcs15_dec}(\text{sk}, c)$ (apply textbook RSA then strip and validate padding, returning $\perp$ on malformed padding).
4. **Determinism attack on textbook RSA:** Show that encrypting the same short message (e.g., a vote or a coin flip) twice produces identical ciphertexts, leaking information. Contrast with PKCS#1 v1.5 where random PS prevents this.
5. **Bleichenbacher padding oracle (simplified):** Implement a toy version with small N (≈ 512 bits): given a ciphertext and a padding-validation oracle, recover the plaintext using the adaptive chosen-ciphertext attack. This demonstrates why PKCS#1 v1.5 is not CCA-secure.
6. **Interface:** Expose $\text{Enc}(\text{pk}, m) \rightarrow c$ and $\text{Dec}(\text{sk}, c) \rightarrow m$ in both textbook and PKCS#1 v1.5 variants, for use in PA#14, PA#15, and PA#18. Your key generation must also output $(p, q, d_p, d_q, q_{\text{inv}})$ for use in PA#14's CRT-based decryption.

PA #12 — Interactive Demo Deliverable

Demo: Textbook RSA determinism attack.

- Student types a short message (e.g., “yes” or “no”, simulating a vote).
- Click “Encrypt twice.” Both ciphertexts are shown — they are identical. A red banner reads “Identical ciphertexts: plaintext leaked.”
- Switch to PKCS#1 v1.5 mode: click “Encrypt twice” again. Ciphertexts differ each time. Green banner.
- A “Padding bytes” panel shows the random PS bytes that differ between the two encryptions.

Toy parameters: 512-bit N for fast in-browser computation. Graders do not need 2048-bit keys for this demo.

4.3 PA #13 — Miller-Rabin Primality Testing

Standalone Concept: Primality Testing

RSA and ElGamal both require large primes (≥ 1024 bits). Deterministic primality testing (AKS) is polynomial but slow. Miller-Rabin is a fast **probabilistic** test: if it says “composite,” it is definitely correct; if it says “prime,” there is at most a 4^{-k} probability of error after k rounds.

Miller-Rabin Algorithm

Input: Odd integer $n > 2$; number of rounds k .

1. Write $n - 1 = 2^s \cdot d$ with d odd.
2. For $i = 1$ to k :
 - (a) Choose $a \leftarrow \{2, \dots, n - 2\}$ at random.

- (b) Compute $x \leftarrow a^d \bmod n$.
- (c) If $x = 1$ or $x = n - 1$: continue.
- (d) For $r = 1$ to $s - 1$:
 - $x \leftarrow x^2 \bmod n$
 - If $x = n - 1$: break (go to next round)
- (e) If $x \neq n - 1$: return COMPOSITE.

3. Return PROBABLY PRIME.

Error probability: $\leq 4^{-k}$ for k rounds. For $k = 40$, probability $\approx 10^{-24}$.

PA #13 — What You Must Implement

1. **Miller-Rabin test:** Implement `millerrabin(n, k)` exactly as described above. Use your own modular exponentiation (square-and-multiply). Return `PROBABLY_PRIME` or `COMPOSITE`.
2. **Prime generation:** Implement `gen_prime(bits)` that repeatedly samples a random odd b -bit integer and tests it with $k = 40$ rounds until a probable prime is found. Verify the output passes 100 rounds of testing as a sanity check.
3. **Carmichael number demo:** Show that $n = 561$ (the smallest Carmichael number) passes a naïve Fermat primality test but is correctly rejected by Miller-Rabin.
4. **Performance benchmark:** Report the average number of candidates sampled before finding a 512-bit prime, a 1024-bit prime, and a 2048-bit prime. Compare to the theoretical $O(\ln n)$ candidates predicted by the Prime Number Theorem.
5. **Interface:** Expose `is_prime(n) -> bool` and `gen_prime(bits) -> int` for use in PA#11 and PA#12.

PA #13 — Interactive Demo Deliverable

Demo: Primality tester.

- A number input field (student types any integer up to 20 digits).
- A “rounds” slider ($k = 1-40$).
- Click “Test.” The app runs Miller-Rabin and shows: `PRIME` or `COMPOSITE`, time taken, and the list of witnesses a_i tested with their computed values at each round.
- Pre-loaded examples: 561 (Carmichael, fooled by Fermat but caught by Miller-Rabin), a known 512-bit prime, and a known composite. Student can click these for instant demo.

Enter 561 — must return `COMPOSITE` after 1+ rounds despite passing Fermat.

4.4 PA #14 — Chinese Remainder Theorem & Breaking Textbook RSA

Intuition

Two roles of CRT in RSA: The Chinese Remainder Theorem plays two opposite roles in RSA. First, it is a *tool for the honest party*—RSA decryption can be made roughly four times faster by working modulo p and q separately (CRT-based RSA). Second, it is a *weapon for the adversary*—if the same message m is broadcast to multiple recipients using a small public exponent e , CRT lets an attacker recover m entirely without knowing any private key. PA#14 implements both.

The Chinese Remainder Theorem

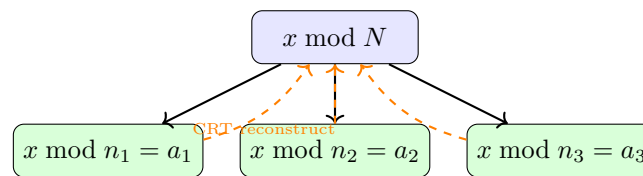
Chinese Remainder Theorem (CRT)

Let $n_1, n_2, \dots, n_k$ be pairwise coprime moduli (i.e., $\gcd(n_i, n_j) = 1$ for $i \neq j$). For any integers $a_1, a_2, \dots, a_k$, the system of congruences

$$x \equiv a_1 \pmod{n_1}, \quad x \equiv a_2 \pmod{n_2}, \quad \dots, \quad x \equiv a_k \pmod{n_k}$$

has a **unique** solution modulo $N = n_1 \cdot n_2 \cdots n_k$:

$$x = \sum_{i=1}^k a_i \cdot M_i \cdot (M_i^{-1} \bmod n_i) \pmod{N} \quad \text{where } M_i = N/n_i$$



Given a_1, a_2, a_3 and n_1, n_2, n_3 coprime $\Rightarrow$ unique $x \bmod N$

Application 1: CRT-Based RSA Decryption (Garner's Algorithm)

Standard RSA decryption computes $C^d \bmod N$ where $N \approx 2^{2048}$ — a single expensive modular exponentiation. Since the private key holder knows p and q , they can instead compute:

$$m_p = C^{d_p} \bmod p, \quad m_q = C^{d_q} \bmod q, \quad \text{then combine via CRT}$$

where $d_p = d \bmod (p-1)$ and $d_q = d \bmod (q-1)$. The exponents are half the size, and the moduli are half the size — giving a $\approx 4\times$ speedup.

Garner's CRT Recombination

Given $m_p = C^{d_p} \bmod p$ and $m_q = C^{d_q} \bmod q$:

1. Let $h = q_{inv} \cdot (m_p - m_q) \bmod p$, where $q_{inv} = q^{-1} \bmod p$.
2. Output $m = m_q + h \cdot q$.

This is equivalent to $C^d \bmod N$ but uses exponentiations modulo p and q individually.

Application 2: Håstad's Broadcast Attack (CRT Breaks Textbook RSA)

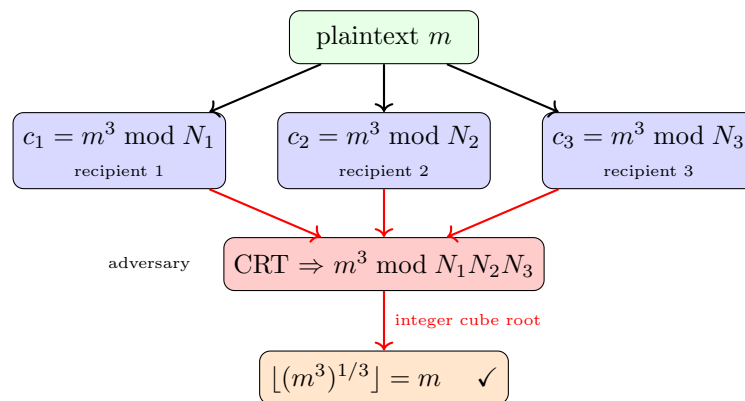
Håstad's Broadcast Attack

Setup: A sender broadcasts the same message m to e different recipients, each with their own RSA modulus N_i but all using the same small public exponent e (e.g., $e = 3$). The adversary sees:

$$c_1 = m^e \bmod N_1, \quad c_2 = m^e \bmod N_2, \quad \dots, \quad c_e = m^e \bmod N_e$$

Attack: Since $m < N_i$ for all i , we have $m^e < N_1 N_2 \cdots N_e$. Apply CRT to recover $m^e \bmod (N_1 N_2 \cdots N_e)$, which equals m^e exactly (as an integer). Then take the e -th integer root to get m .

For $e = 3$: Three ciphertexts suffice. CRT gives $x = m^3$ as an integer; compute $\lfloor x^{1/3} \rfloor$. **This requires no factoring and no private keys.** It defeats textbook RSA completely when e is small and m is short.



Why This Attack Fails on Padded RSA

With PKCS#1 v1.5 or OAEP, the message padded before encryption includes random bytes, so each recipient encrypts a *different* padded value. The three ciphertexts no longer share a common m , so CRT recovers garbage, and the integer cube root step fails.

PA #14 — What You Must Implement

- CRT solver:** Implement `crt(residues, moduli) -> x` that takes a list of (a_i, n_i) pairs (with pairwise coprime n_i) and returns the unique $x \bmod \prod n_i$ satisfying all congruences. Use the constructive formula with modular inverses. Implement `mod_inverse(a, n)` yourself via the extended Euclidean algorithm (no library).
- CRT-based RSA decryption (Garner's algorithm):** Implement `rsa_dec_crt(sk, c)` where `sk` includes (p, q, d_p, d_q, q_inv) . Compute $m_p = c^{d_p} \bmod p$, $m_q = c^{d_q} \bmod q$, then recombine using Garner's formula. Verify correctness: `rsa_dec_crt(sk, c) == rsa_dec(sk, c)` for 100 random messages.
- Performance comparison:** Benchmark `rsa_dec` (standard) vs. `rsa_dec_crt` (Garner) for 1000 decryptions each at 1024-bit and 2048-bit key sizes. Report the speedup ratio and confirm it is $\approx 3\text{--}4\times$.
- Håstad's broadcast attack:** Implement `hastad_attack(ciphertexts, moduli, e)` where `ciphertexts[i] = m^e mod moduli[i]` for $i = 0, \dots, e - 1$. Steps:

- (a) Apply your CRT solver to recover $x = m^e \bmod \prod N_i$.
- (b) Compute the integer e -th root of x (implement Newton's method for integer roots).
- (c) Return m .

Test with $e = 3$: generate three independent RSA key pairs $(N_1, 3), (N_2, 3), (N_3, 3)$, encrypt the same short message m to all three, and recover m using your attack. Confirm m is recovered exactly.

5. **Attack boundary:** Determine the maximum message length (in bytes) for which Håstad's attack with $e = 3$ succeeds, given three 1024-bit moduli. Explain why messages with $m^3 \geq N_1 N_2 N_3$ are safe from this specific attack (though still insecure for other reasons).
6. **Padding defeats the attack:** Show that if each sender applies PKCS#1 v1.5 padding (from PA#12) before encrypting, the broadcast attack fails — the three padded plaintexts differ, so CRT does not recover m^3 and the integer root step returns garbage.
7. **Interface:** Expose `crt(residues, moduli) -> int`, `rsa_dec.crt(sk, c) -> int`, and `hastad_attack(ciphertexts, moduli, e) -> int` for use in PA#17 (which exploits this to motivate CCA-secure PKC).

PA #14 — Interactive Demo Deliverable

Demo: Håstad broadcast attack visualiser.

- Three recipient panels, each showing their modulus N_i and a “received ciphertext” $c_i = m^3 \bmod N_i$.
- An “Attacker” panel below runs CRT on the three ciphertexts live and displays the recovered integer m^3 .
- Click “Cube root.” The integer cube root is computed and the plaintext m is revealed — matching the original.
- A “Use PKCS padding” toggle re-runs the attack with padded ciphertexts. The attack fails: CRT still works but the cube root is not an integer, and the padded garbage is displayed.

Toy parameters: 64-bit N_i for instant computation. Graders check that the plaintext is recovered in unpadded mode and fails in padded mode.

4.5 PA #15 — Digital Signatures

Intuition

Digital signatures are the public-key analogue of MACs. Alice signs a message with her *private key*; anyone can verify with her *public key*. Unlike MACs, signatures provide **non-repudiation**—Alice cannot deny having signed a message, because only she has the private key.

Digital Signature Definition

(Gen, Sign, Vrfy) satisfying:

- $\text{Gen}(1^n) \rightarrow (vk, sk)$: verification key and signing key.
- $\text{Sign}_{sk}(m) \rightarrow \sigma$: signature on m .
- $\text{Vrfy}_{vk}(m, \sigma) \rightarrow \{0, 1\}$: 1 iff valid.

Security (EUF-CMA): No PPT adversary with access to signing oracle can forge a valid (m^*, σ^*) for any m^* not previously signed.

RSA Signature (textbook): $\sigma = m^d \bmod N$, verify: $\sigma^e \stackrel{?}{=} m \bmod N$.

In practice: Always sign the hash of the message: $\sigma = H(m)^d \bmod N$.

PA #15 — What You Must Implement

1. **RSA signature scheme:** Implement $\text{sign}(sk, m) = H(m)^d \bmod N$ and $\text{verify}(vk, m, \text{sigma}) = (\sigma^e \bmod N \stackrel{?}{=} H(m))$. Use your PA#12 RSA and your PA#8 hash. Signing raw m without hashing is not acceptable (vulnerable to existential forgery).
2. **Hash-then-sign:** Explicitly hash the message before signing. Demonstrate that without hashing, an adversary can forge a signature on $m_1 \cdot m_2$ given signatures on m_1 and m_2 (multiplicative homomorphism attack on raw RSA).
3. **EUF-CMA game:** Implement the signing oracle game. Show that after seeing up to 50 signed messages, the adversary cannot produce a valid (m^*, σ^*) for any new m^* (with overwhelming probability).
4. **ElGamal signature (alternative):** Optionally also implement Schnorr or ElGamal signatures using your PA#16 group, providing a DLP-based alternative to RSA-based signing.
5. **Interface:** Expose $\text{Sign}(sk, m) \rightarrow \text{sigma}$ and $\text{Verify}(vk, m, \text{sigma}) \rightarrow \text{bool}$ for use in PA#17.

PA #15 — Interactive Demo Deliverable

Demo: Sign and verify live.

- A text input for a message and a “Sign” button. The signature $\sigma = H(m)^d \bmod N$ is displayed in hex.
- A “Verify” button checks $\sigma^e \bmod N \stackrel{?}{=} H(m)$, showing the intermediate values and a green “Valid” or red “Invalid” result.
- A “Tamper” button flips one bit of the message after signing; verification immediately fails.
- “Raw RSA sign (no hash)” toggle: signs m directly without hashing, then demonstrates the multiplicative forgery — given signatures on m_1 and m_2 , compute a valid signature on $m_1 \cdot m_2$ without the private key.

4.6 PA #16 — ElGamal Public-Key Cryptosystem

Intuition

ElGamal is a PKC based directly on the hardness of DLP. Unlike RSA (which needs new number-theoretic assumptions), ElGamal security reduces cleanly to DDH (Decisional Diffie-Hellman). Its structure closely mirrors the DH key exchange.

ElGamal Cryptosystem

Setup: Cyclic group $\mathbb{G}$ of prime order q with generator g .

Key Generation:

1. Sample private key $x \leftarrow \mathbb{Z}_q$.
2. Set public key $h = g^x$.

Encryption of $m \in \mathbb{G}$:

1. Sample $r \leftarrow \mathbb{Z}_q$.
2. Output $C = (g^r, m \cdot h^r) = (g^r, m \cdot g^{xr})$.

Decryption of (c_1, c_2) :

1. Compute $m = c_2 / c_1^x = m \cdot g^{xr} / g^{rx} = m$.

Security Claim

ElGamal is CPA-secure under the DDH assumption. The ciphertext $(g^r, m \cdot h^r)$ is indistinguishable from (g^r, random) when DDH holds.

Warning: Standard ElGamal is also malleable (like textbook RSA)—it is only CPA-secure, not CCA-secure.

PA #16 — What You Must Implement

1. **Key generation:** Implement `elgamal_keygen()` using your PA#11 group (prime p , generator g , order q). Sample $x \leftarrow \mathbb{Z}_q$, compute public key $h = g^x \bmod p$. Output $(\text{sk}=x, \text{pk}=(p, g, q, h))$.
2. **Encryption and decryption:** Implement `ElGamal_Enc(pk, m)` — sample fresh $r \leftarrow \mathbb{Z}_q$, return $(g^r \bmod p, m \cdot h^r \bmod p)$ — and `ElGamal_Dec(sk, c1, c2) = c2 / c1^x \bmod p`.
3. **Malleability attack:** Given ciphertext (c_1, c_2) for unknown m , construct $(c_1, 2c_2 \bmod p)$ which decrypts to $2m$, without knowing m or x . Show this breaks CCA security.
4. **CPA game:** Implement the IND-CPA game for ElGamal. Confirm the adversary advantage is ≈ 0 when DDH is hard (large group), and demonstrate a distinguisher succeeds when the group is tiny (e.g., $q \approx 2^{10}$).
5. **Interface:** Expose `Enc(pk, m) -> (c1, c2)` and `Dec(sk, c1, c2) -> m` for use in PA#17 and PA#18.

PA #16 — Interactive Demo Deliverable

Demo: ElGamal malleability.

- Encrypt a plaintext group element m (entered as an integer). Ciphertext (c_1, c_2) is

shown.

- A “Multiply c_2 by 2” button produces $(c_1, 2c_2 \bmod p)$ — a new valid ciphertext. Click “Decrypt.” The result $2m$ is shown, demonstrating malleability.
- A counter shows how many times this trick succeeds (should be 100%).
- A side panel shows the CPA game: encrypt a challenge, submit a modified ciphertext to the decryption oracle, confirm $2m$ comes back, demonstrating why ElGamal fails CCA.

Confirm that $\text{Dec}(c_1, 2c_2) = 2 \cdot \text{Dec}(c_1, c_2)$ for any message.

4.7 PA #17 — CCA-Secure PKC

Intuition

Textbook RSA and basic ElGamal are malleable: given $C = (g^r, m \cdot h^r)$, one can produce $C' = (g^r, 2m \cdot h^r)$ encrypting $2m$ without knowing m . CCA security defeats this. We combine Digital Signatures (PA#15) with PKC (PA#16) in the **Sign-then-Encrypt**-then-MAC or **Fujisaki-Okamoto** paradigm.

Signcryption Construction (PA#15 + PA#16)

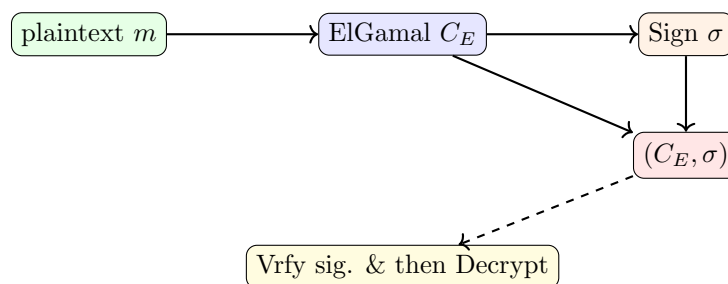
Encrypt: $C_E \leftarrow \text{ElGamal}_{pk}(m)$

Sign: $\sigma \leftarrow \text{Sign}_{sk}(C_E)$

Output: (C_E, σ)

Decrypt:

1. Verify $\text{Vrfy}_{vk}(C_E, \sigma) = 1$ (reject if not)
2. Output $\text{ElGamal.Dec}_{sk}(C_E)$



PA #17 — What You Must Implement

1. **Signcrypt (Encrypt-then-Sign):** Implement `CCA_PKC_Enc(pk_enc, sk_sign, m)`: first encrypt m with your PA#16 ElGamal (or PA#12 RSA), then sign the ciphertext with your PA#15 digital signature. Return (C_E, σ) .
2. **Verify-then-Decrypt:** Implement `CCA_PKC_Dec(sk_enc, vk_sign, C_E, sigma)`: call `Verify` first; if the signature is invalid return $\perp$; otherwise call `Dec`. The signature check *must precede* decryption — this ordering is not optional.
3. **CCA2 game for PKC:** Implement the IND-CCA2 game with a decryption oracle. Show that your scheme correctly rejects modified ciphertexts (the signature fails), so

the decryption oracle is useless to the adversary.

4. **Contrast with CPA-only:** Demonstrate the malleability attack from PA#16 on plain ElGamal: the adversary submits a modified ciphertext to the decryption oracle and gets back $2m$. Show the same attack fails on your PA#17 scheme because the signature is invalid on the modified ciphertext.
5. **Full lineage check:** Your submission must trace the entire dependency chain in code: PA#17 calls PA#15 (your signatures) and PA#16 (your ElGamal), which call PA#12 and PA#11, which call PA#13. No library substitutions at any layer.

PA #17 — Interactive Demo Deliverable

Demo: CCA malleability blocked.

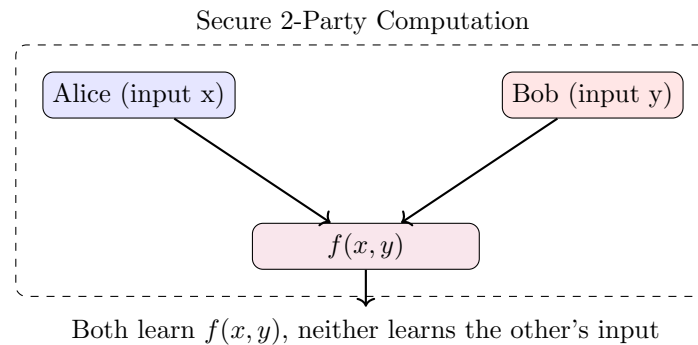
- Encrypt a message using Encrypt-then-Sign. The ciphertext (C_E, σ) is shown.
- A “Tamper with C_E ” button (simulating the CCA attacker) modifies one byte of C_E .
- Submit to decryption oracle. The app shows signature verification firing first — “Signature invalid, decryption aborted, output $\perp$.” The plaintext never decrypts.
- Contrast panel (plain ElGamal): the same tamper goes through, and $2m$ is returned.

Any tampered ciphertext returns $\perp$. Untampered ciphertext decrypts correctly.

5 Part IV: Secure Multi-Party Computation (MPC)

Intuition

The ultimate goal: Two mutually distrusting parties Alice and Bob hold private inputs x and y . Can they jointly compute $f(x, y)$ *without* revealing their inputs to each other? Yao's Garbled Circuits and the GMW protocol say: **yes**, and the foundation is **Oblivious Transfer (OT)**.



5.1 PA #18 — Oblivious Transfer (OT)

Lineage Requirement

The PKC used in this OT implementation **must be your own implementation from PA#12 (RSA) or PA#16 (ElGamal)**. Importing any external PKC library violates the no-library rule and breaks the reduction chain.

Standalone Concept: Oblivious Transfer

1-out-of-2 OT: Sender has bits (m_0, m_1) . Receiver has choice bit $b \in \{0, 1\}$. The OT protocol ensures:

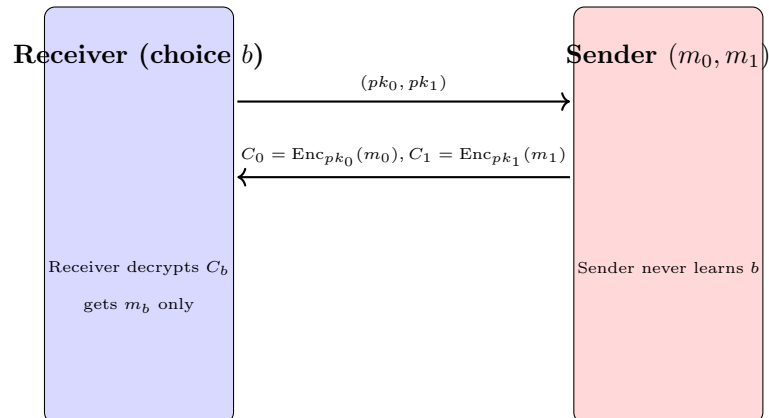
- Receiver learns m_b and *nothing* about m_{1-b} .
- Sender learns *nothing* about b .

This is the **atomic building block** of all MPC.

OT from PKC (Bellare-Micali)

Setup: PKC scheme (Gen, Enc, Dec).

1. Receiver generates **two key pairs**: (pk_0, sk_0) and (pk_1, sk_1) , but only keeps sk_b (discards sk_{1-b} , or constructs pk_{1-b} without knowing its secret key using a trap-door).
2. Receiver sends both (pk_0, pk_1) to Sender.
3. Sender encrypts: sends $C_0 = \text{Enc}_{pk_0}(m_0)$ and $C_1 = \text{Enc}_{pk_1}(m_1)$.
4. Receiver decrypts $C_b = \text{Dec}_{sk_b}(C_b)$ to get m_b ; cannot decrypt C_{1-b} .



PA #18 — What You Must Implement

- 1-out-of-2 OT from your PKC**: Implement the full Bellare-Micali OT protocol using your own PA#16 ElGamal (or PA#12 RSA). Your implementation must have two clearly separated roles:
 - OT_Receiver_Step1(b) $\rightarrow$ (pk_0, pk_1, state)**: Receiver generates pk_b honestly (keeping sk_b) and constructs pk_{1-b} without a trapdoor (e.g., by choosing a random group element as the public key, for which no private key is known). Returns both public keys and private state.
 - OT_Sender_Step(pk_0, pk_1, m_0, m_1) $\rightarrow$ (C_0, C_1)**: Sender encrypts both messages and returns both ciphertexts.
 - OT_Receiver_Step2(state, C_0, C_1) $\rightarrow m_b$** : Receiver decrypts only C_b using sk_b .
- Receiver privacy**: Demonstrate that the sender, given (pk_0, pk_1) , cannot determine b — because pk_{1-b} was constructed without a trapdoor, both keys are computationally indistinguishable.
- Sender privacy**: Demonstrate that the receiver cannot decrypt C_{1-b} — they have no sk_{1-b} . For small parameters, attempt a brute-force decryption and show it requires solving DLP or factoring.
- Correctness**: Run 100 trials with random $b \in \{0, 1\}$ and random (m_0, m_1) . Verify that the receiver always recovers m_b correctly.
- Interface**: Expose the three-step API above so PA#19 can call OT as a black box with no knowledge of the underlying PKC.

PA #18 — Interactive Demo Deliverable

Demo: Play the OT receiver. The student is Bob:

- Alice's panel (left, greyed out): holds two messages m_0 and m_1 , hidden.
- Bob's panel (right, interactive): student clicks "Choose 0" or "Choose 1."
- The OT protocol runs step-by-step with a message log showing: key pairs generated, (pk_0, pk_1) sent to Alice, C_0 and C_1 received, C_b decrypted.
- Result: m_b is revealed to Bob. m_{1-b} remains hidden (shown as "??"). A "Cheat attempt" button tries to decrypt C_{1-b} and shows the failure.

Choose 0 and 1 in separate runs; confirm correct message received each time and the other is hidden.

5.2 PA #19 — Secure AND Gate

Intuition

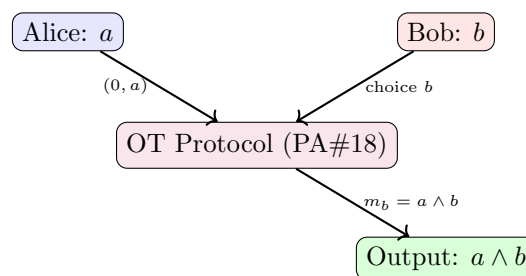
If we can evaluate AND securely, we can evaluate any boolean function securely (since AND and XOR form a **functionally complete** basis). The key trick: use OT to implement AND without either party revealing their input bit.

Secure AND from OT

Alice holds $a \in \{0, 1\}$, Bob holds $b \in \{0, 1\}$. Goal: both learn $a \wedge b$.

1. Alice acts as *OT sender* with messages $(m_0, m_1) = (0, a)$.
2. Bob acts as *OT receiver* with choice bit b .
3. Bob receives $m_b = a \cdot b = a \wedge b$.

Security: Bob learns only $a \wedge b$ (not a). Alice learns nothing about b (OT guarantee).



PA #19 — What You Must Implement

1. **Secure AND from OT:** Implement `Secure_AND(a, b)` using your PA#18 OT as a subroutine. Alice plays OT sender with messages $(m_0, m_1) = (0, a)$; Bob plays OT receiver with choice bit b ; Bob receives $m_b = a \wedge b$. Both parties then locally output $a \wedge b$.
2. **Secure XOR (free):** Implement `Secure_XOR(a, b)`: this requires *no* OT call. Alice and Bob each locally hold a share; the XOR of their shares equals the result. Implement additive secret sharing over $\mathbb{Z}_2$: Alice sends $r \leftarrow \{0, 1\}$ to Bob; Alice's share is $a \oplus r$, Bob's share is $b \oplus r$; the output is the XOR of both shares. No information about either party's input is revealed.
3. **NOT gate (free):** Implement `Secure_NOT(a)`: Alice locally flips her share. No communication needed.
4. **Privacy proof (informal):** For `Secure_AND`, write a brief argument (code comment or README) showing: (a) Bob learns nothing about a beyond $a \wedge b$ (follows from OT receiver privacy); (b) Alice learns nothing about b (follows from OT sender privacy).
5. **Truth table test:** Verify all four input combinations $(a, b) \in \{00, 01, 10, 11\}$ produce the correct AND and XOR outputs across 50 runs each. Confirm no party can recover the other's input bit from the protocol transcript alone.
6. **Interface:** Expose `AND(a, b) -> bit` and `XOR(a, b) -> bit` so that PA#20 can compose them into arbitrary circuits.

PA #19 — Interactive Demo Deliverable

Demo: Secure AND step-by-step.

- Alice panel: student enters bit $a \in \{0, 1\}$.
- Bob panel: student enters bit $b \in \{0, 1\}$.
- Click “Compute AND.” A step-log shows: Alice sets up OT messages $(0, a)$, Bob runs OT receiver with choice b , Bob receives $m_b = a \wedge b$.
- The transcript (all messages exchanged) is shown. A “What does Alice learn?” and “What does Bob learn?” summary confirms neither party sees the other’s input.
- All four (a, b) combinations have a “Run all” button that confirms correct AND output for each.

Run all 4 combinations; verify outputs match AND truth table and transcript reveals nothing extra.

5.3 PA #20 — All 2-Party Secure Computation (Yao / GMW)

Lineage Requirement

PA#20 must be built using **your own PA#19 (Secure AND)** and **your own PA#18 (OT)**, which in turn uses **your own PKC from PA#12/#16**. The entire MPC stack must trace back to your own implementations with no library substitutions at any layer.

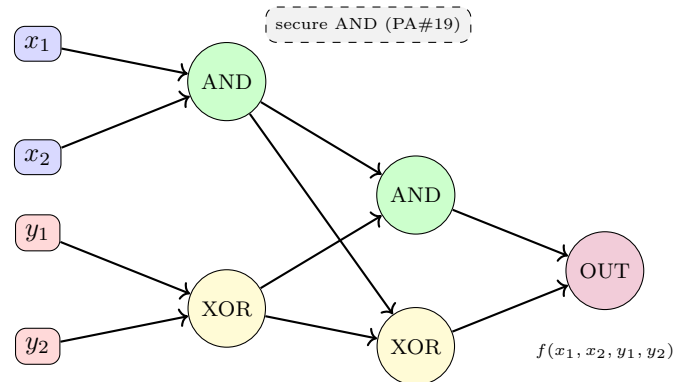
The Grand Theorem: MPC Completeness

Given Secure AND (PA#19) and Secure XOR (free via additive secret sharing), we can securely evaluate **any** polynomial-time computable 2-party function $f(x, y)$.

Why? Any boolean circuit can be expressed using AND and XOR gates (NOT is free). Compose our secure gates to get a secure circuit. This is the essence of Yao’s Garbled Circuits.

Yao’s Garbled Circuit (Sketch)

1. Alice *garbles* the circuit C computing f : for each gate, she creates an encrypted truth table using OT-based keys.
2. Alice sends the garbled circuit $\tilde{C}$ and her input labels to Bob.
3. Bob uses OT (PA#18) to get labels for *his* input bits without Alice learning Bob’s input.
4. Bob evaluates $\tilde{C}$ gate by gate (in topological order) and learns the output.



Security Claim

Using Secure AND (from OT, from PKC) and Secure XOR (free), PA#20 implements any boolean circuit securely. This completes the full cryptographic stack: from OWF at the base to arbitrary secure computation at the top.

PA #20 — What You Must Implement

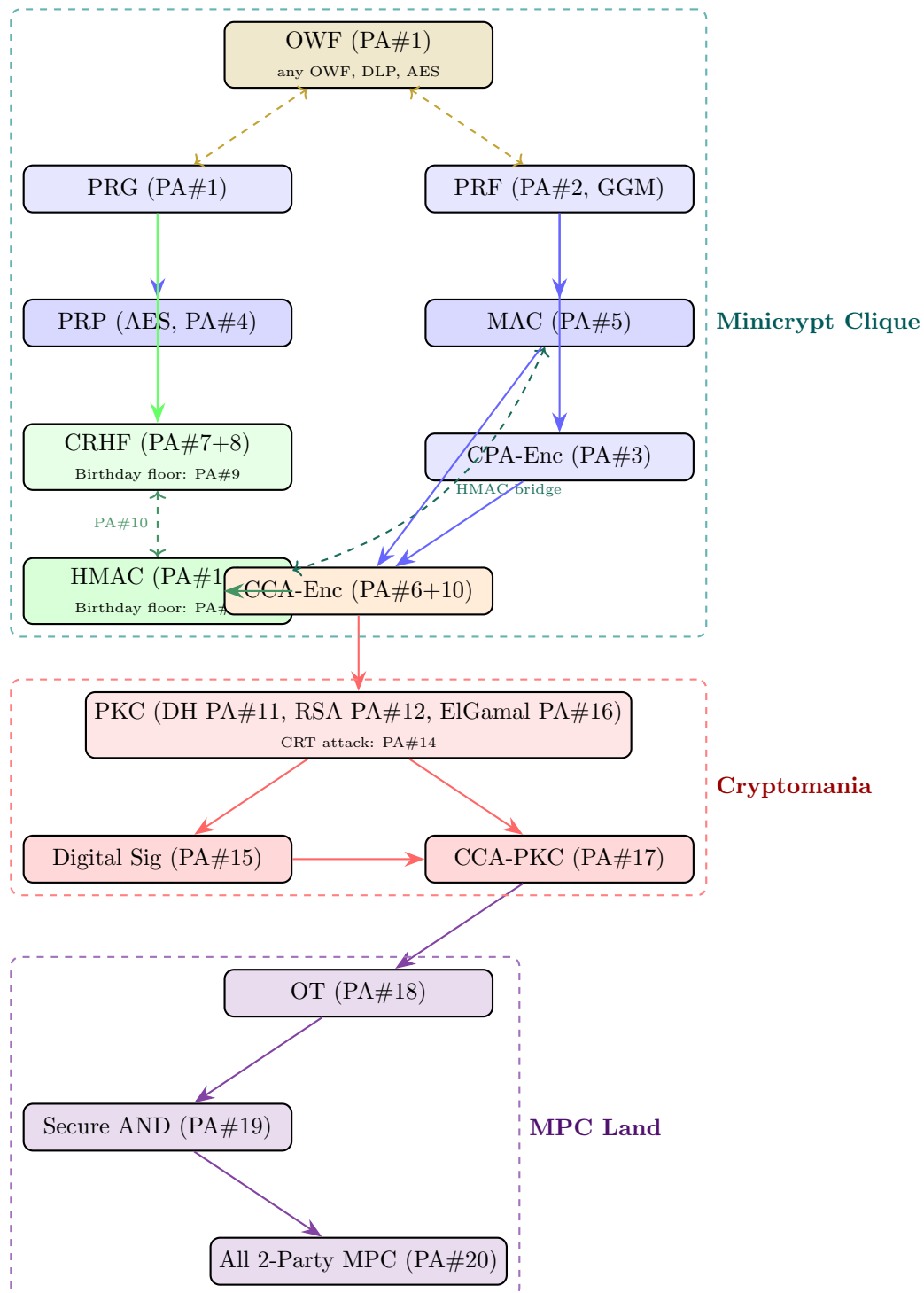
1. **Boolean circuit evaluator:** Implement a `Circuit` class that represents any boolean function as a directed acyclic graph (DAG) of AND, XOR, and NOT gates. Nodes store gate type and wire indices; the circuit takes a list of input bits and evaluates to output bits.
2. **Secure circuit evaluation:** Implement `Secure_Eval(circuit, x_Alice, y_Bob)` which evaluates the circuit on Alice's input x and Bob's input y using **only** your PA#19 AND, XOR, and NOT gate primitives. Traverse the circuit in topological order; call the corresponding secure gate for each node.
3. **Three mandatory test circuits:** You must implement and securely evaluate the following:
 - **Millionaire's Problem:** Alice has integer x , Bob has integer y . Securely compute $x > y$ (who is richer) without revealing the actual values. Implement the comparison circuit over n -bit integers using AND and XOR gates.
 - **Secure equality test:** Compute $x = y$ (bit-wise equality of n -bit integers) without revealing x or y .
 - **Secure bit-addition:** Compute the sum $x + y$ of two n -bit integers modulo 2^n , revealing only the result.
4. **Privacy verification:** For each of the three circuits above, confirm that neither party learns anything beyond the output. Concretely: log all messages exchanged during the protocol; show that the transcript is simulatable (i.e., it could have been generated from the output alone, without the other party's input).
5. **End-to-end lineage:** The evaluation of a single AND gate must ultimately trigger: PA#19 AND $\rightarrow$ PA#18 OT $\rightarrow$ PA#16 ElGamal (or PA#12 RSA) $\rightarrow$ PA#13 Miller-Rabin (for key generation). Provide a call-stack trace in your README demonstrating this full chain for at least one gate evaluation.
6. **Performance:** Report the number of OT calls required to evaluate each of the three circuits above, and the total wall-clock time for $n = 8$ -bit inputs. This gives a concrete sense of the cost of MPC.

PA #20 — Interactive Demo Deliverable**Demo: Millionaire's problem live.**

- Alice panel: slider for her wealth x (1–100, hidden from Bob's panel).
- Bob panel: slider for his wealth y (1–100, hidden from Alice's panel).
- Click “Who is richer?” The comparison circuit evaluates gate by gate; a progress bar shows gates completed.
- Result: “Alice is richer” / “Bob is richer” / “Equal” — displayed to both. The actual values x and y are never revealed.
- A “Circuit trace” expandable section shows the AND/XOR gates evaluated and the output wire values (but *not* the input wires of the other party).

Toy parameters: $n = 4$ -bit comparison (16 possible values). Fast enough to animate gate-by-gate. Set $x = 7, y = 12$; confirm Bob is richer without Alice's panel revealing 7.

6 Summary: The Full Reduction Chain



Assignment Index

PA	Topic	Must use your own impl. of	Bidir.
#0	Minicrypt Clique Web Explorer (React)	Stubs PA#1–#10 (incremental)	
#1	OWF + PRG	—	✓
#2	PRF (GGM tree)	#1 (or self-impl. AES)	✓
#3	CPA-secure encryption	#2	
#4	Modes of Operation (CBC/OFB/CTR)	#3	
#5	Secure MACs (PRF-MAC, CBC-MAC)	#2 (or self-impl. AES)	
#6	CCA-secure encryption	#3 + #5	
#7	Merkle-Damgrd Transform	—	
#8	DLP-based CRHF	#7	
#9	Birthday Attack (collision finding)	#8 (truncated)	
#10	HMAC + Encrypt-then-HMAC	#8 + #3	✓
#11	Diffie-Hellman SKE	#13	
#12	Textbook RSA + PKCS#1 v1.5	#13	
#13	Miller-Rabin Primality	—	
#14	CRT + Håstad Broadcast Attack	#12 + #13	
#15	Digital Signatures	#12 or #16	
#16	ElGamal PKC	#11	
#17	CCA-Secure PKC	#15 + #16	
#18	Oblivious Transfer	#12 or #16 (your PKC!)	
#19	Secure AND	#18	
#20	All 2-party MPC	#19 + #18	

Bidir. (✓) = bidirectional reduction required (both $A \Rightarrow B$ and $B \Rightarrow A$ must be implemented). PA#1–#2 cover the core Minicrypt clique; PA#10 adds the CRHF $\leftrightarrow$ MAC bridge via HMAC. All other PAs require only a forward construction.